University of Sheffield

A World TOP 100 University

WHAT UNI? STUDENT CHOICE Awards 2024 UNIVERSITY OF THE YEAR

Regression Testing

Software Testing (COM3529)

Checkin code: LO-TN-KJ

José Rojas & Neil Walkinshaw

Week 6

Roadmap

Beizer's Maturity Model

Test Automation

Unit Testing

Control / Data-Flow Analysis

Code Coverage

Regression Testing

Model-Based Testing

Mutation Testing

Search-Based Test Generation

Fuzzing

Roadmap

Beizer's Maturity Model

Test Automation

Unit Testing

Control / Data-Flow Analysis

Code Coverage

Regression Testing

Model-Based Testing

Mutation Testing

Search-Based Test Generation

Fuzzing

What is Regression Testing?

Motivation

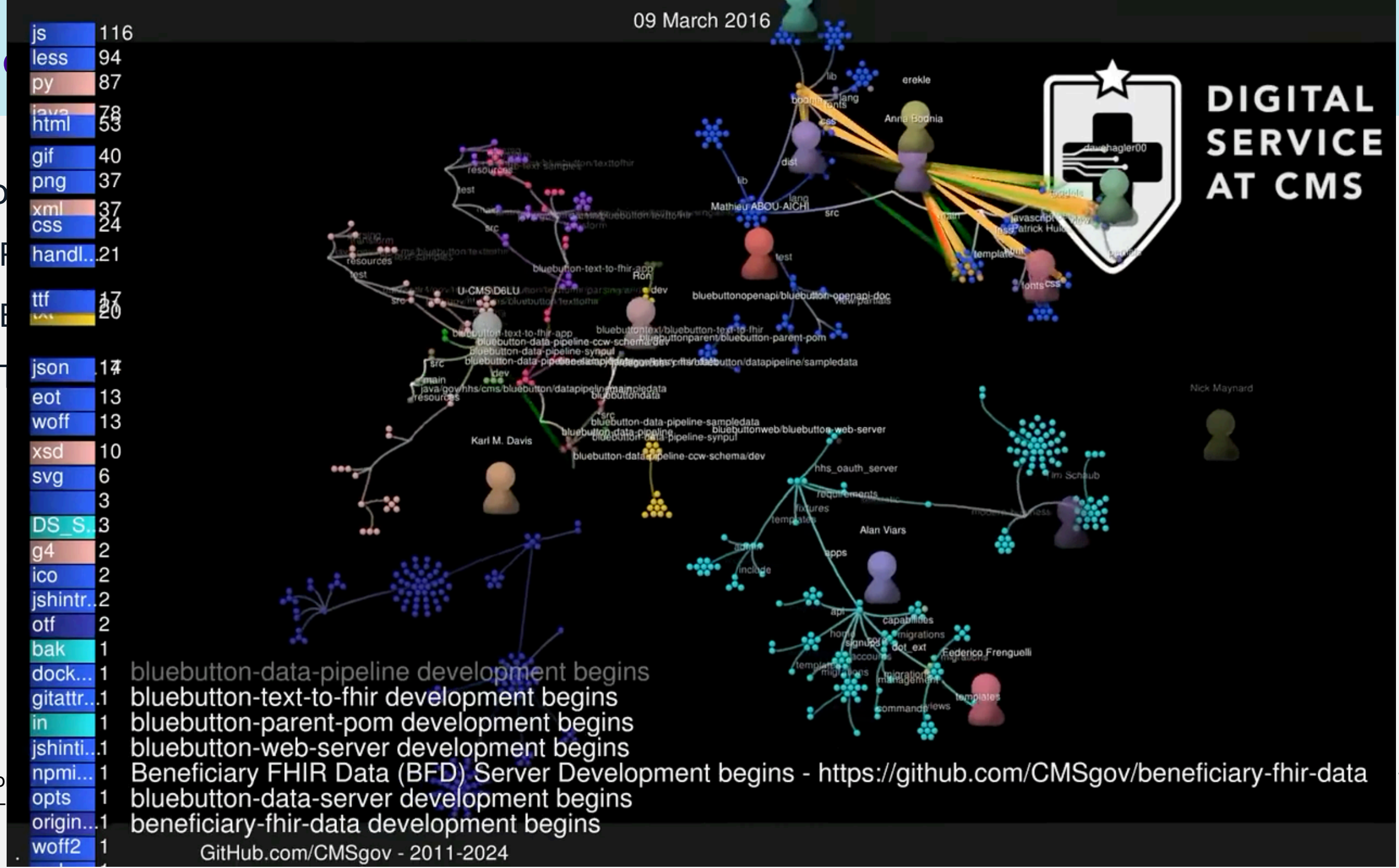
Software is not a *static* entity. It is **continuously subject to changes**.

Requirements change.

Bugs are fixed.

The environment changes - regulatory changes, devices, operating platforms, etc.

Memon, Atif, et al. "Taming Google-scale continuous testing." 2017 IEEE/ACM 39th International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP). IEEE, 2017.


09 March 2016
DIGITAL SERVICE AT CMS
js 116
less 94
py 87
java 78
html 53
gif 40
png 37
xml 37
css 24
handl... 21
ttf 17
json 14
eot 13
woff 13
xsd 10
svg 6
3
DS_S... 3
g4 2
ico 2
jshintr... 2
otf 2
bak 1
dock... 1
gitattr... 1
in 1
jshinti... 1
npmi... 1
opts 1
origin... 1
woff2 1
bluebutton-data-pipeline development begins
bluebutton-text-to-fhir development begins
bluebutton-parent-pom development begins
bluebutton-web-server development begins
Beneficiary FHIR Data (BFD) Server Development begins - https://github.com/CMSgov/beneficiary-fhir-data
bluebutton-data-server development begins
beneficiary-fhir-data development begins
GitHub.com/CMSgov - 2011-2024
Mathieu ABOU-AICHI
Anna Bodnia
erekle
Karl M. Davis
Alan Viars
Federico Frenguelli
Nick Maynard

Motivation

Software is not a *static* entity. It is **continuously subject to changes**.

Requirements change.

Bugs are fixed.

The environment changes - regulatory changes, devices, operating platforms, etc.

Memon, Atif, et al. "Taming Google-scale continuous testing." 2017 IEEE/ACM 39th International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP). IEEE, 2017.

Motivation

Software is not a *static* entity. It is **continuously subject to changes**.

- Requirements change.
- Bugs are fixed.
- The environment changes - regulatory changes, devices, operating platforms, etc.

Need to ensure that changes to the system do not result in **regressions**.

- Unexpected reductions in software quality.

Memon, Atif, et al. "Taming Google-scale continuous testing." 2017 IEEE/ACM 39th International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP). IEEE, 2017.

Motivation

Software is not a *static* entity. It is **continuously subject to changes**.

- Requirements change.
- Bugs are fixed.
- The environment changes - regulatory changes, devices, operating platforms, etc.

Need to ensure that changes to the system do not result in **regressions**.

- Unexpected reductions in software quality.

Test sets can be **time and resource-consuming to run**.

- Developers at Google wait between **45 minutes and 9 hours** to receive test feedback.

Memon, Atif, et al. "Taming Google-scale continuous testing." 2017 IEEE/ACM 39th International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP). IEEE, 2017.

Motivation

Software is not a *static* entity. It is **continuously subject to changes**.

- Requirements change.
- Bugs are fixed.
- The environment changes - regulatory changes, devices, operating platforms, etc.

Need to ensure that changes to the system do not result in **regressions**.

- Unexpected reductions in software quality.

Test sets can be **time and resource-consuming to run**.

- Developers at Google wait between **45 minutes and 9 hours** to receive test feedback.

Memon, Atif, et al. "Taming Google-scale continuous testing." 2017 IEEE/ACM 39th International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP). IEEE, 2017.

Regression Testing

What is regression testing?

Testing with the specific aim of **detecting regressions**.

Often automated within a CI/CD (Continuous Integration & Development) framework.

Regression Testing

What is regression testing?

- Testing with the specific aim of **detecting regressions**.
- **Often automated** within a CI/CD (Continuous Integration & Development) framework.

Based on assumption that an **existing test-suite** exists.

- E.g. a suite of automated unit- and integration-tests developed alongside the core system.

Regression Testing

What is regression testing?

- Testing with the specific aim of **detecting regressions**.
- **Often automated** within a CI/CD (Continuous Integration & Development) framework.

Based on assumption that an **existing test-suite** exists.

- E.g. a suite of automated unit- and integration-tests developed alongside the core system.

Challenge is to manage the scale of these existing test suites.

Regression Testing

What is regression testing?

- Testing with the specific aim of **detecting regressions**.
- **Often automated** within a CI/CD (Continuous Integration & Development) framework.

Based on assumption that an **existing test-suite** exists.

- E.g. a suite of automated unit- and integration-tests developed alongside the core system.

Challenge is to manage the scale of these existing test suites.

Miranda, Charles, Guilherme Avelino, and Pedro Santos Neto. "Test Co-Evolution in Software Projects: A Large-Scale Empirical Study." Journal of Software: Evolution and Process 37.7 (2025): e70035.

Regression Testing

What is regression testing?

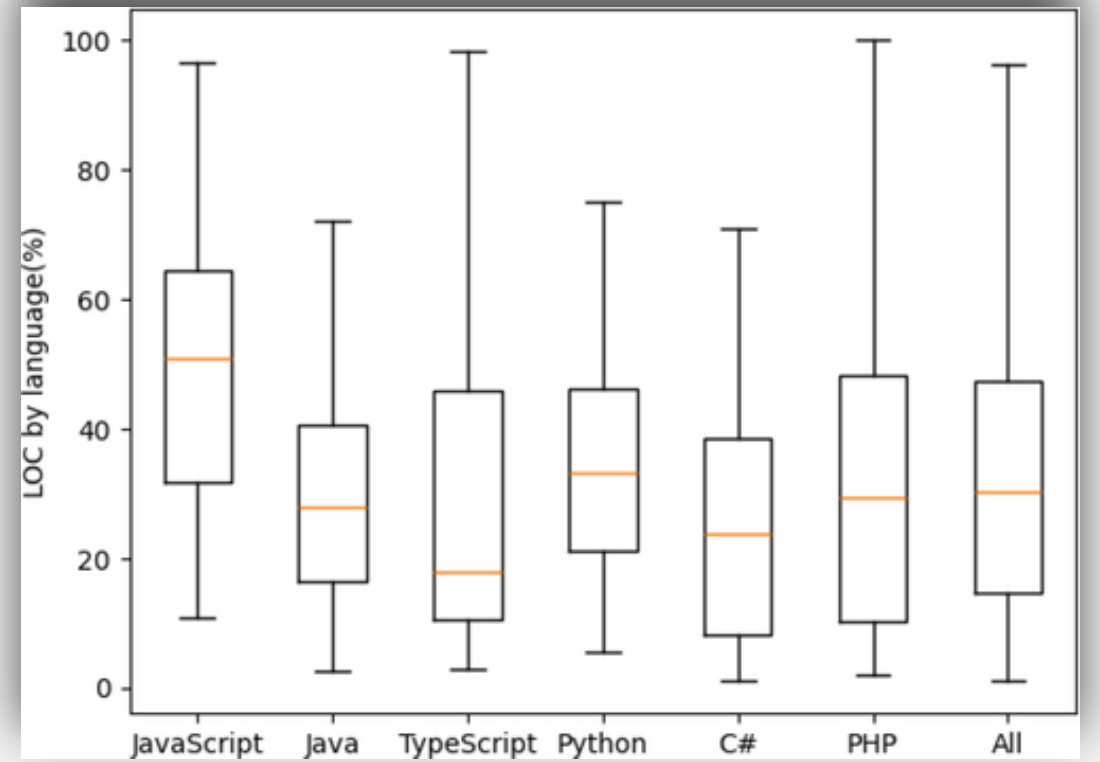
- Testing with the specific aim of **detecting regressions**.
- **Often automated** within a CI/CD (Continuous Integration & Development) framework.

Based on assumption that an **existing test-suite** exists.

- E.g. a suite of automated unit- and integration-tests developed alongside the core system.

Challenge is to manage the scale of these existing test suites.

Tests as proportion of source code



Miranda, Charles, Guilherme Avelino, and Pedro Santos Neto. "Test Co-Evolution in Software Projects: A Large-Scale Empirical Study." Journal of Software: Evolution and Process 37.7 (2025): e70035.

Regression Testing

What is regression testing?

Testing with the specific aim of **detecting regressions**.

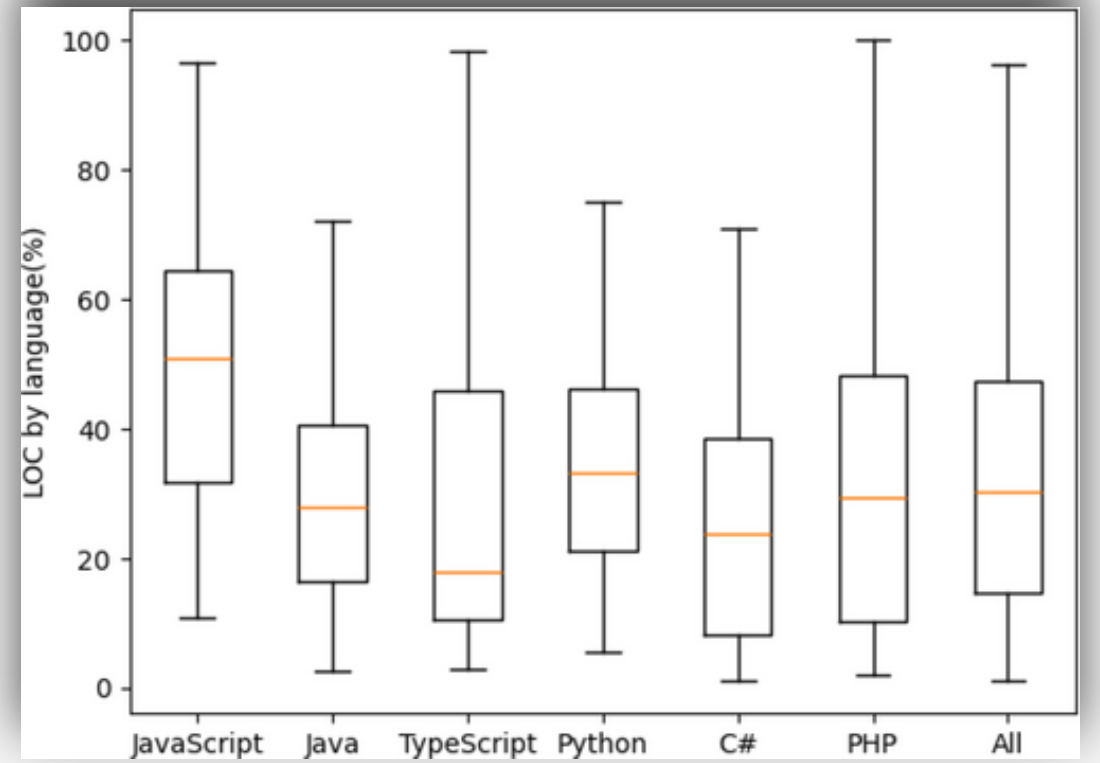
Often automated within a CI/CD (Continuous Integration & Development) framework.

Based on assumption that an **existing test-suite** exists.

E.g. a suite of automated unit- and integration-tests developed alongside the core system.

Challenge is to manage the scale of these existing test suites.

Tests as proportion of source code



Sampled from 526 GitHub projects

In >80% of projects, test suits outgrow application code.

Miranda, Charles, Guilherme Avelino, and Pedro Santos Neto. "Test Co-Evolution in Software Projects: A Large-Scale Empirical Study." Journal of Software: Evolution and Process 37.7 (2025): e70035.

Key Regression Testing Challenges

Minimisation: How can I reduce the number of tests I need to run

Key Regression Testing Challenges

Minimisation: How can I reduce the number of tests I need to run

Prioritisation: In which order should I run my tests, in order to detect defects as soon as possible?

Key Regression Testing Challenges

Minimisation: How can I reduce the number of tests I need to run

Prioritisation: In which order should I run my tests, in order to detect defects as soon as possible?

Selection: Given a specific code change, which tests should I select to focus on this change?

Test Minimisation

Rationale

In a continuously growing test suite, many tests will become **redundant**.

- Superseded by newer tests.
- Testing old features that have since been removed or updated.

Rationale

In a continuously growing test suite, many tests will become **redundant**.

- Superseded by newer tests.
- Testing old features that have since been removed or updated.

How can we **remove this redundancy**?

- We have access to coverage information from previously executed CI/CD tests.
  - E.g. statement coverage, branch coverage, …
- We know which code has changed, and where we need to focus our tests on.

Test Minimisation

Given a test suite, find the smallest subset to maintain the coverage of the original.

- There are several variants of this challenge.
- e.g. finding some T' that also minimises time or resource required to execute.

Tend to rely on assumption that *preserving coverage preserves the effectiveness* of T'

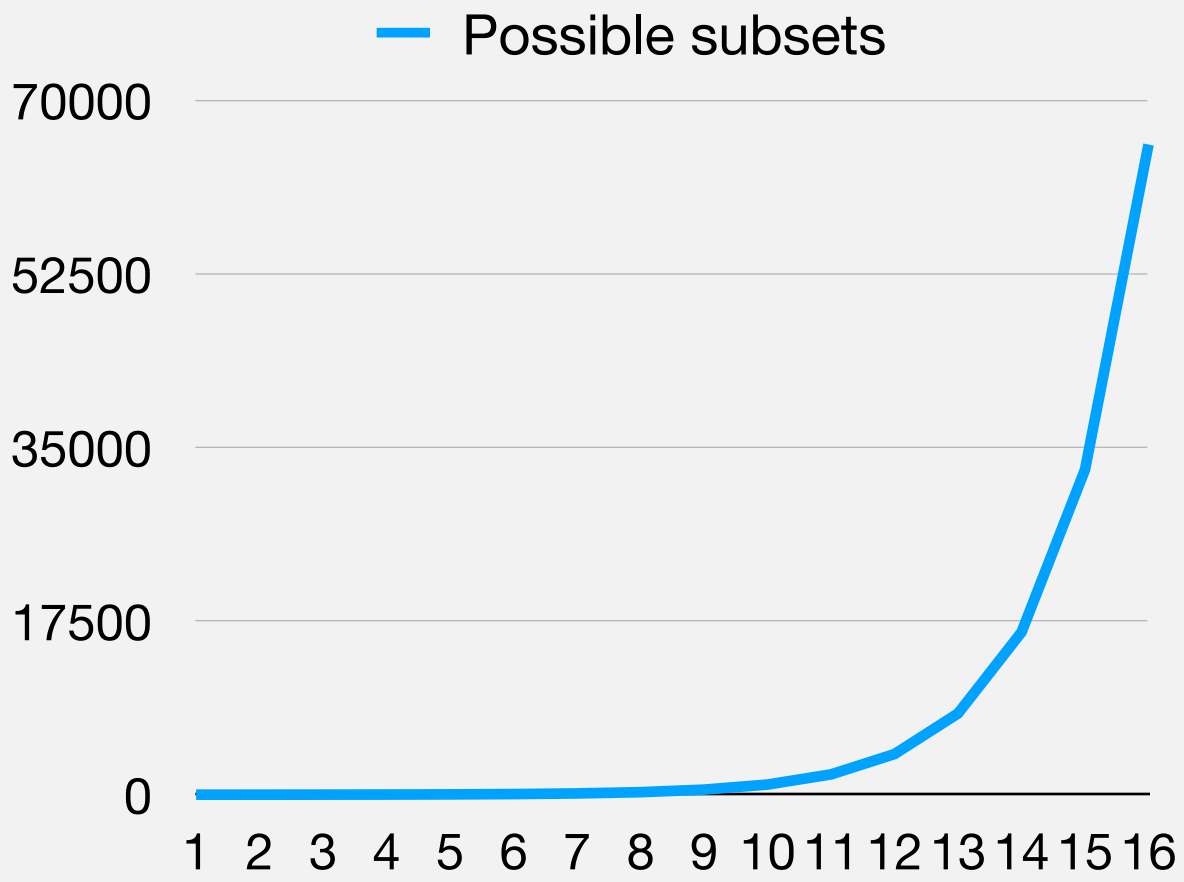
- *Why might this assumption not hold?*

Formalisation:

- Let us consider our coverage goals (e.g. statements) as a set $C = \{c_1, \ldots, c_k\}$.
- Consider an existing test suite $T = \{t_1, \ldots, t_n\}$, where for each test t_i, $coverage(t_i) \subseteq C$
- The goal is to find the smallest $T' \subseteq T$, such that $\bigcup_{t_i \in T'} coverage(t_i) = \bigcup_{t_i \in T} coverage(t_i)$

Test Minimisation is NP-complete

Number of sub-sets grows exponentially with number of test cases.

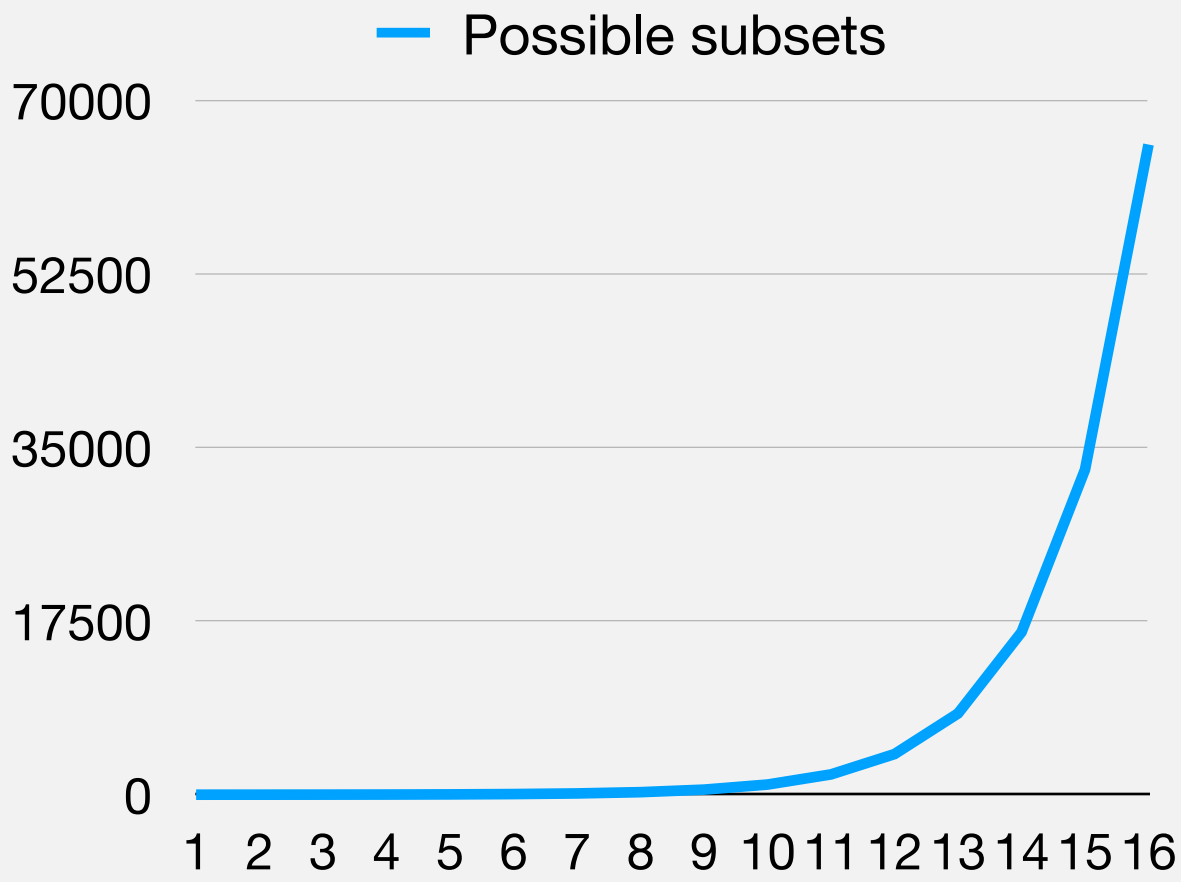


Test Minimisation is NP-complete

Number of sub-sets grows exponentially with number of test cases.

For n tests, there are 2^n possible subsets to consider.

- For just 16 test cases we need to consider 65,536 subsets!
- Becomes intractable for realistic systems.



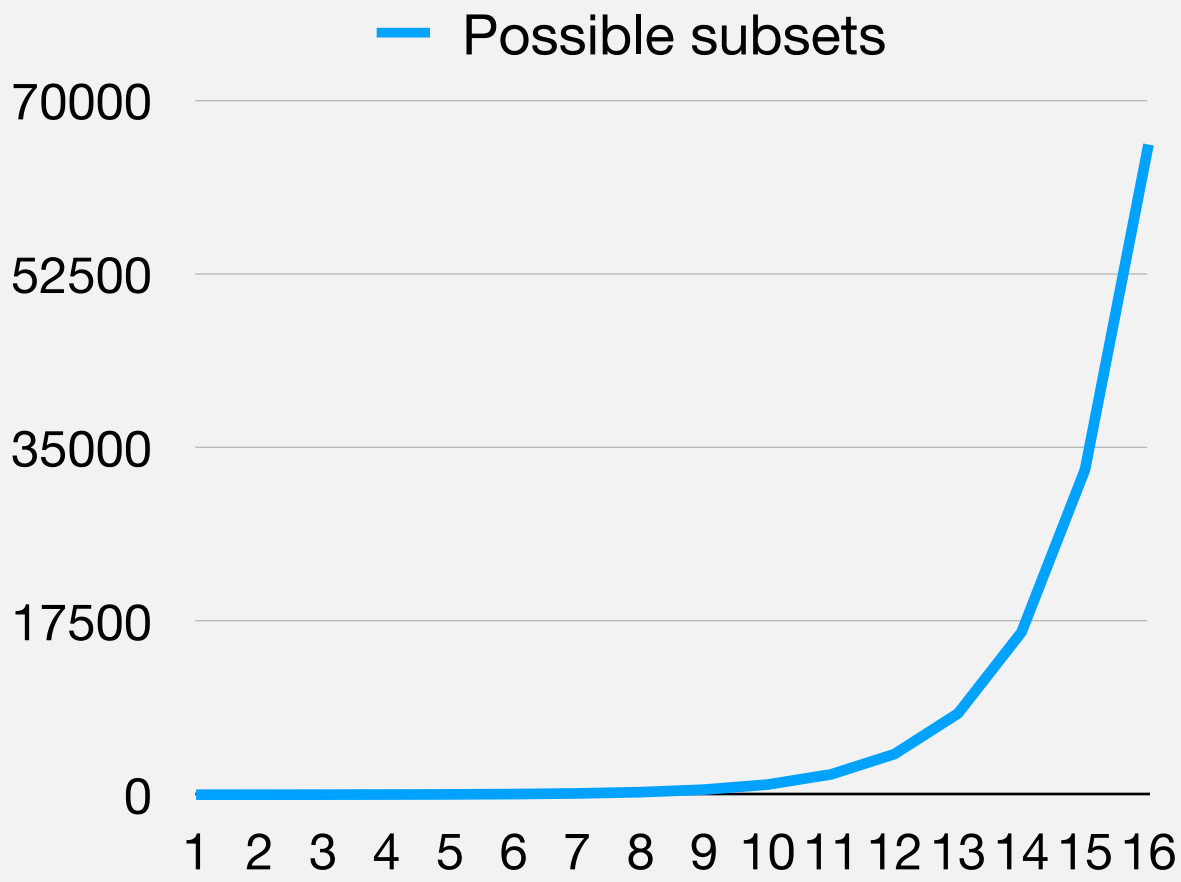
Test Minimisation is NP-complete

Number of sub-sets grows exponentially with number of test cases.

For n tests, there are 2^n possible subsets to consider.

- For just 16 test cases we need to consider 65,536 subsets!
- Becomes intractable for realistic systems.

Necessary to adopt heuristic inexact approaches.



A Simple Greedy Algorithm

Create a pool of potential tests $P = T$

Start with $T' = \emptyset$.

Identify a test $t_i \in P$ that covers the most goals.

Add t_i to T' and remove it from P.

Iteratively:

Add to T' the next test $t_j \in P$ that covers the most goals, and remove from P.

Terminate when $\bigcup_{t_i \in T'} coverage(t_i) = \bigcup_{t_i \in T} coverage(t_i)$

Simple Greedy Algorithm Example

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	X	X	X			X	X	X
B	X	X	X				X	X
C	X	X	X	X				
D					X	X	X	X
E	X	X	X			X	X	X

$T' := \{\}$

Simple Greedy Algorithm Example

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x				x	x
C	x	x	x	x				
D					x	x	x	x
E	x	x	x			x	x	x

$T' := \{\}$

Start with **A** or **E** (either order)

Simple Greedy Algorithm Example

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x				x	x
C	x	x	x	x				
D					x	x	x	x
E	x	x	x			x	x	x

T':= {A,E}

Start with **A** or **E** (either order)

Simple Greedy Algorithm Example

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x				x	x
C	x	x	x	x				
D					x	x	x	x
E	x	x	x			x	x	x

$T' := \{A, E\}$

Start with **A** or **E** (either order)
Next we choose **B**

Simple Greedy Algorithm Example

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x				x	x
C	x	x	x	x				
D					x	x	x	x
E	x	x	x			x	x	x

T':= {A,E,B}

Start with **A** or **E** (either order)
Next we choose **B**

Simple Greedy Algorithm Example

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x				x	x
C	x	x	x	x				
D					x	x	x	x
E	x	x	x			x	x	x

T':= {A,E,B}

Start with **A** or **E** (either order)
Next we choose **B**
Finally **C** and **D** (either order)

Simple Greedy Algorithm Example

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x				x	x
C	x	x	x	x				
D					x	x	x	x
E	x	x	x			x	x	x

T':= {A,E,B,C,D}

Start with **A** or **E** (either order)
Next we choose **B**

Simple Greedy Algorithm Example

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x				x	x
C	x	x	x	x				
D					x	x	x	x
E	x	x	x			x	x	x

T':= {A,E,B,C,D}

Start with **A** or **E** (either order)
Next we choose **B**
Finally **C** and **D** (either order)

Simple Greedy Algorithm Example

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x				x	x
C	x	x	x	x				
D					x	x	x	x
E	x	x	x			x	x	x

T':= {A,E,B,C,D}

Start with **A** or **E** (either order)
Next we choose **B**
Finally **C** and **D** (either order)

But could have just been C and D!

Additional Greedy

Take into account the **coverage already achieved**!

Create a pool of potential tests $P = T$

Start with $T' = \emptyset$.

Identify a test $t_i \in P$ that covers the most goals.

Add t_i to T' and remove it from P.

Iteratively:

Add to T' the next test $t_j \in P$ that covers *the most currently uncovered* goals, and remove from P.

Terminate when $\bigcup_{t_i \in T'} coverage(t_i) = \bigcup_{t_i \in T} coverage(t_i)$

Additional Greedy Algorithm Example

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	X	X	X			X	X	X
B	X	X	X				X	X
C	X	X	X	X				
D					X	X	X	X
E	X	X	X			X	X	X

$T' := \{\}$

Additional Greedy Algorithm Example

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x				x	x
C	x	x	x	x				
D					x	x	x	x
E	x	x	x			x	x	x

$T' := \{\}$

Start with **A** or **E** (either order)

Simple Greedy Algorithm Example

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x				x	x
C	x	x	x	x				
D					x	x	x	x
E	x	x	x			x	x	x

$T' := \{A\}$

Start with **A** or **E** (either one)

b4 and b5 are uncovered.

Simple Greedy Algorithm Example

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x				x	x
C	x	x	x	x				
D					x	x	x	x
E	x	x	x			x	x	x

$T' := \{A\}$

Start with **A** or **E** (either one)

b4 and b5 are uncovered.

Add **C** and **D** (either order)

Simple Greedy Algorithm Example

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x				x	x
C	x	x	x	x				
D					x	x	x	x
E	x	x	x			x	x	x

T':= {A,C,D}

Start with **A** or **E** (either one)

b4 and b5 are uncovered.

All goals covered!

Simple Greedy Algorithm Example

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x				x	x
C	x	x	x	x				
D					x	x	x	x
E	x	x	x			x	x	x

T':= {A,C,D}

Start with **A** or **E** (either one)

b4 and b5 are uncovered.

Add **C** and **D** (either order)

All goals covered!

Simple Greedy Algorithm Example

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x				x	x
C	x	x	x	x				
D					x	x	x	x
E	x	x	x			x	x	x

T':= {A,C,D}

Start with **A** or **E** (either one)

b4 and b5 are uncovered.

Add **C** and **D** (either order)

All goals covered!

Better, but still not optimal (no need for A)

Harrold et al.'s approach (1993)

Prioritise tests that cover **uniquely-covered goals** first.

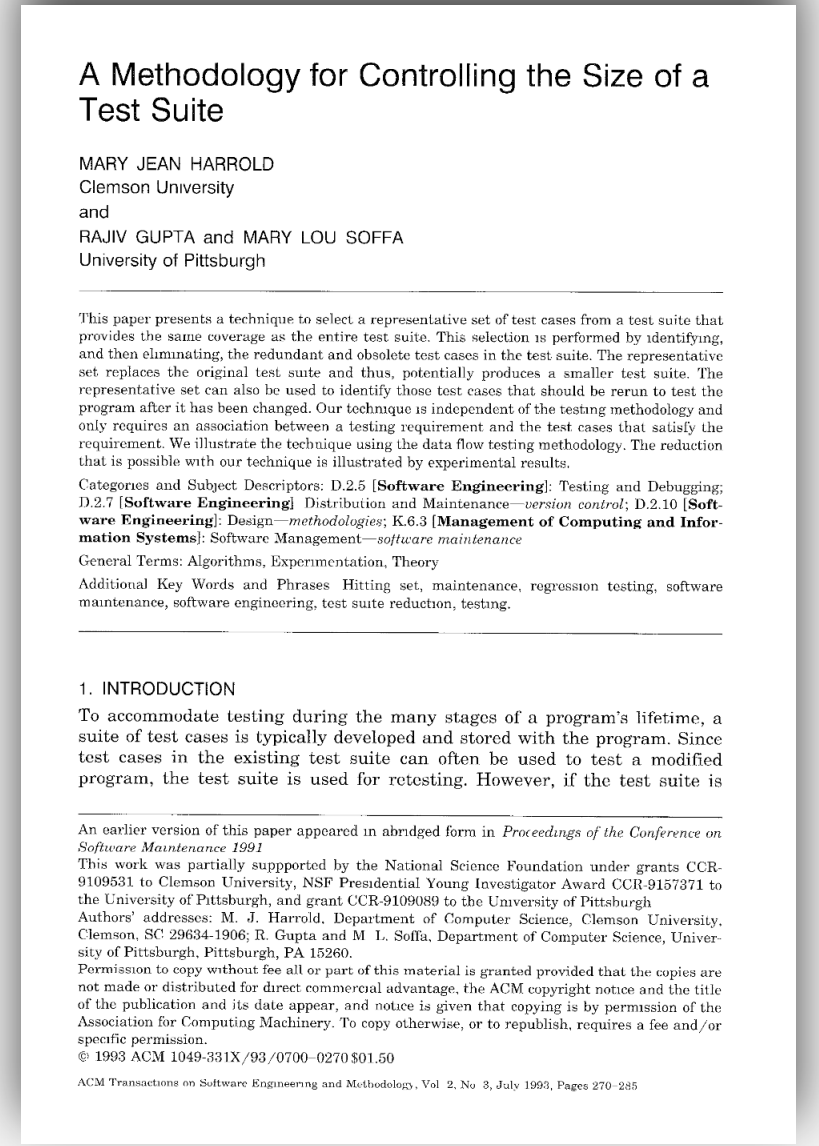
Start by building a mapping from each coverage goal to the set of tests that cover it.

We can denote this map $HIT : C \rightarrow T$

E.g. $HIT(b_1) = \{A, B, E\}$

We care about the cardinality of the set for each target.

The number of tests in a set - so $|HIT(b_1)| = 3$



A Methodology for Controlling the Size of a Test Suite

MARY JEAN HARROLD
Clemson University
and
RAJIV GUPTA and MARY LOU SOFFA
University of Pittsburgh

This paper presents a technique to select a representative set of test cases from a test suite that provides the same coverage as the entire test suite. This selection is performed by identifying, and then eliminating, the redundant and obsolete test cases in the test suite. The representative set replaces the original test suite and thus, potentially produces a smaller test suite. The representative set can also be used to identify those test cases that should be rerun to test the program after it has been changed. Our technique is independent of the testing methodology and only requires an association between a testing requirement and the test cases that satisfy the requirement. We illustrate the technique using the data flow testing methodology. The reduction that is possible with our technique is illustrated by experimental results.

Categories and Subject Descriptors: D.2.5 [**Software Engineering**]: Testing and Debugging; D.2.7 [**Software Engineering**] Distribution and Maintenance—*version control*; D.2.10 [**Software Engineering**]: Design—*methodologies*; K.6.3 [**Management of Computing and Information Systems**]: Software Management—*software maintenance*

General Terms: Algorithms, Experimentation, Theory

Additional Key Words and Phrases: Hitting set, maintenance, regression testing, software maintenance, software engineering, test suite reduction, testing.

1. INTRODUCTION

To accommodate testing during the many stages of a program's lifetime, a suite of test cases is typically developed and stored with the program. Since test cases in the existing test suite can often be used to test a modified program, the test suite is used for retesting. However, if the test suite is

An earlier version of this paper appeared in abridged form in *Proceedings of the Conference on Software Maintenance 1991*
This work was partially suppported by the National Science Foundation under grants CCR-9109531 to Clemson University, NSF Presidential Young Investigator Award CCR-9157371 to the University of Pittsburgh, and grant CCR-9109089 to the University of Pittsburgh
Authors' addresses: M. J. Harrold, Department of Computer Science, Clemson University, Clemson, SC 29634-1906; R. Gupta and M. L. Soffa, Department of Computer Science, University of Pittsburgh, Pittsburgh, PA 15260.

Harrold et al.'s approach (1993)

Start with $T' = \emptyset$.

For $i \in 1, \ldots, |T|$:

Identify all testing targets $c \in C$ such that $|HIT(c)| = i$

For each identified c:

$T' = T' \cup HIT(c)$

Finish if $\bigcup_{t_i \in T'} coverage(t_i) = \bigcup_{t_i \in T} coverage(t_i)$

Harrold et al. Example

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x				x	x
C	x	x	x	x				
D					x	x	x	x

$$i = 1$$

$$T' := \{\}$$

$$HIT(b4) = \{C\}$$

$$HIT(b5) = \{D\}$$

Harrold et al. Example

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x				x	x
C	x	x	x	x				
D					x	x	x	x
E	x	x	x			x	x	x

$$i = 1$$

$$T' := \{\}$$

$$HIT(b4) = \{C\}$$

$$HIT(b5) = \{D\}$$

Harrold et al. Example

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x				x	x
C	x	x	x	x				
D					x	x	x	x
E	x	x	x			x	x	x
Tests	A,B, C,E	A,B, C,E	A,B, C,E	C	D	A,D, E	A,B, D,E	A,B, D,E

$$i = 1$$

$$T' := \{\}$$

$$HIT(b4) = \{C\}$$

$$HIT(b5) = \{D\}$$

Harrold et al. Example

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x				x	x
C	x	x	x	x				
D					x	x	x	x

$i = 1$

T':= {C,D}

Harrold et al. Example

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x				x	x
C	x	x	x	x				
D					x	x	x	x
E	x	x	x			x	x	x

$i = 1$

T':= {C,D}

Harrold et al. Example

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x				x	x
C	x	x	x	x				
D					x	x	x	x
E	x	x	x			x	x	x
Tests	A,B, C,E	A,B, C,E	A,B, C,E	C	D	A,D, E	A,B, D,E	A,B, D,E

$i = 1$

T':= {C,D}

Harrold et al. Example

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x				x	x
C	x	x	x	x				
D					x	x	x	x
E	x	x	x			x	x	x
Tests	A,B, C,E	A,B, C,E	A,B, C,E	C	D	A,D, E	A,B, D,E	A,B, D,E

$i = 1$

T':= {C,D}

Works nicely here - full coverage.

But what if there are ties?
I.e. the same block is covered by multiple tests?

Harrold et al. Example (altered to include ties)

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x		x		x	x
C	-	-	-	x				
D					x	x	x	x

$$i = 1$$

$$T' := \{\}$$

$$HIT(b4) = \{C\}$$

Harrold et al. Example (altered to include ties)

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x		x		x	x
C	-	-	-	x				
D					x	x	x	x
E	x	x	x			x	x	x

$$i = 1$$

$$T' := \{\}$$

$$HIT(b4) = \{C\}$$

Harrold et al. Example (altered to include ties)

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x		x		x	x
C	-	-	-	x				
D					x	x	x	x
E	x	x	x			x	x	x
Tests	A,B, C,E	A,B, C,E	A,B, C,E	C	B,D	A,D, E	A,B, D,E	A,B, D,E

$$i = 1$$

$$T' := \{\}$$

$$HIT(b4) = \{C\}$$

Harrold et al. Example (altered to include ties)

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x		x		x	x
C				x				
D					x	x	x	x

$i = 2$

T':= {C}

$HIT(b5) = \{B, D\}$

Harrold et al. Example (altered to include ties)

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x		x		x	x
C				x				
D					x	x	x	x
E	x	x	x			x	x	x

$i = 2$

T':= {C}

$HIT(b5) = \{B, D\}$

Harrold et al. Example (altered to include ties)

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x		x		x	x
C				x				
D					x	x	x	x
E	x	x	x			x	x	x
Tests	A,B, C,E	A,B, C,E	A,B, C,E	C	B,D	A,D, E	A,B, D,E	A,B, D,E

$$i = 2$$

$$T' := \{C\}$$

$$HIT(b5) = \{B, D\}$$

Harrold et al. Example (altered to include ties)

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x		x		x	x
C				x				
D					x	x	x	x
E	x	x	x			x	x	x
Tests	A,B, C,E	A,B, C,E	A,B, C,E	C	B,D	A,D, E	A,B, D,E	A,B, D,E

$$i = 2$$

$$T' := \{C\}$$

$$HIT(b5) = \{B, D\}$$

Which one should we choose?

Harrold et al. Example (altered to include ties)

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x		x		x	x
C				x				
D					x	x	x	x
E	x	x	x			x	x	x
Tests	A,B, C,E	A,B, C,E	A,B, C,E	C	B,D	A,D, E	A,B, D,E	A,B, D,E

$$i = 2$$

$$T' := \{C\}$$

$$HIT(b5) = \{B, D\}$$

Which one should we choose?

Harrold et al. Example (altered to include ties)

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x		x		x	x
C				x				
D					x	x	x	x
E	x	x	x			x	x	x
Tests	A,B, C,E	A,B, C,E	A,B, C,E	C	B,D	A,D, E	A,B, D,E	A,B, D,E

$$i = 2$$

$$T' := \{C\}$$

$$HIT(b5) = \{B, D\}$$

Which one should we choose?

Choose test with most blocks covered by 3 (i+1) tests. Then 4 (i+2), etc.

Harrold et al. Example (altered to include ties)

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x		x		x	x
C				x				
D					x	x	x	x
E	x	x	x			x	x	x
Tests	A,B, C,E	A,B, C,E	A,B, C,E	C	B,D	A,D, E	A,B, D,E	A,B, D,E

$$i = 2$$

$$T' := \{C\}$$

$$HIT(b5) = \{B, D\}$$

Which one should we choose?

Choose test with most blocks covered by 3 (i+1) tests. Then 4 (i+2), etc.

Harrold et al. Example (altered to include ties)

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x		x		x	x
C				x				
D					x	x	x	x
E	x	x	x			x	x	x
Tests	A,B, C,E	A,B, C,E	A,B, C,E	C	B,D	A,D, E	A,B, D,E	A,B, D,E

$$i = 2$$

$$T' := \{C\}$$

$$HIT(b5) = \{B, D\}$$

Which one should we choose?

Choose test with most blocks covered by 3 (i+1) tests. Then 4 (i+2), etc.

For 3 tests:

Harrold et al. Example (altered to include ties)

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x		x		x	x
C				x				
D					x	x	x	x
E	x	x	x			x	x	x
Tests	A,B, C,E	A,B, C,E	A,B, C,E	C	B,D	A,D, E	A,B, D,E	A,B, D,E

$$i = 2$$

$$T' := \{C\}$$

$$HIT(b5) = \{B, D\}$$

Which one should we choose?

Choose test with most blocks covered by 3 (i+1) tests. Then 4 (i+2), etc.

For 3 tests:
B: b1, b2, b3 vs **D: b6**; so add **B**.

Harrold et al. Example (altered to include ties)

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x		x		x	x
C				x				
D					x	x	x	x

$i = 3$

T':= {C,B}

$HIT(b6) = \{A, D, E\}$

Harrold et al. Example (altered to include ties)

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x		x		x	x
C				x				
D					x	x	x	x
E	x	x	x			x	x	x

$i = 3$

T':= {C,B}

$HIT(b6) = \{A, D, E\}$

Harrold et al. Example (altered to include ties)

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x		x		x	x
C				x				
D					x	x	x	x
E	x	x	x			x	x	x
Tests	A,B, C,E	A,B, C,E	A,B, C,E	C	B,D	A,D, E	A,B, D,E	A,B, D,E

$i = 3$

T':= {C,B}

$HIT(b6) = \{A, D, E\}$

Harrold et al. Example (altered to include ties)

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x		x		x	x
C				x				
D					x	x	x	x
E	x	x	x			x	x	x
Tests	A,B, C,E	A,B, C,E	A,B, C,E	C	B,D	A,D, E	A,B, D,E	A,B, D,E

$i = 3$

T':= {C,B}

$HIT(b6) = \{A, D, E\}$

Final minimised suite:

Harrold et al. Example (altered to include ties)

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x		x		x	x
C				x				
D					x	x	x	x
E	x	x	x			x	x	x
Tests	A,B, C,E	A,B, C,E	A,B, C,E	C	B,D	A,D, E	A,B, D,E	A,B, D,E

$i = 3$

T':= {C,B}

$HIT(b6) = \{A, D, E\}$

Final minimised suite:

Harrold et al. Example (altered to include ties)

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8
A	x	x	x			x	x	x
B	x	x	x		x		x	x
C				x				
D					x	x	x	x
E	x	x	x			x	x	x
Tests	A,B, C,E	A,B, C,E	A,B, C,E	C	B,D	A,D, E	A,B, D,E	A,B, D,E

$i = 3$

T':= {C,B}

$HIT(b6) = \{A, D, E\}$

Final minimised suite:

{C,B,A|D|E}

Prioritisation

Rationale

Even if we have minimised the test set…

Rationale

Even if we have minimised the test set…

… in a **time-bound setting**, we may only be able to execute a subset of our test-suite.

Some tests are more likely to expose a fault than others.

In which order should we execute tests to maximise the likelihood of exposing a fault?

We do not know in advance which tests will fail!

Rationale

Even if we have minimised the test set…

… in a **time-bound setting**, we may only be able to execute a subset of our test-suite.

- Some tests are more likely to expose a fault than others.
- In which order should we execute tests to maximise the likelihood of exposing a fault?
- We do not know in advance which tests will fail!

How can we establish an order?

- We know the code-coverage for each test case.
- We know how often test cases have failed in the past.
- We can **prioritise tests that are most likely to lead to failures**!

General Prioritisation Approach

Pick a metric / heuristic for how likely a test is to expose a fault.

- Code coverage.
- Historical data (e.g. how often a test has failed for previous versions).
- …

Order and run tests by the chosen metric.

General Prioritisation Approach

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8	Previous failures
A	x	x	x			x	x	x	13
B	x	x	x				x	x	10
C	x	x	x	x					1
D					x	x	x	x	8
E	x	x	x			x	x	x	13
F	x		x	x				x	6

General Prioritisation Approach

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8	Previous failures
A	x	x	x			x	x	x	13
B	x	x	x				x	x	10
C	x	x	x	x					1
D					x	x	x	x	8
E	x	x	x			x	x	x	13
F	x		x	x				x	6

Ordering by Added Greedy: {A,C,D,B,E,F}

General Prioritisation Approach

Test \ Branch	b1	b2	b3	b4	b5	b6	b7	b8	Previous failures
A	x	x	x			x	x	x	13
B	x	x	x				x	x	10
C	x	x	x	x					1
D					x	x	x	x	8
E	x	x	x			x	x	x	13
F	x		x	x				x	6

Ordering by Added Greedy: {A,C,D,B,E,F}

Ordering by Previous Failures: {A,E,B,D,F,C}

How do we evaluate these different strategies?

Pick a system with ***known* faults**.

These can also be synthesised using an approach called Mutation Testing (to be covered later).

Test \ Faults	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	X	X	X	X											
B					X	X	X								
C					X	X	X	X	X	X		X			
D														X	
E			X	X	X					X		X			
F					X	X	X	X				X	X		

Plotting on a Curve

Test \ Faults	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	X	X	X	X											
B					X	X	X								
C					X	X	X	X	X	X		X			
D														X	
E			X	X	X					X		X			
F					X	X	X	X				X	X		

Plotting on a Curve

Test \ Faults	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	X	X	X	X											
B					X	X	X								
C					X	X	X	X	X	X		X			
D														X	
E			X	X	X					X		X			
F					X	X	X	X				X	X		

Ordering by Added Greedy: {A,C,D,B,E,F}

Plotting on a Curve

Test \ Faults	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	X	X	X	X											
B					X	X	X								
C					X	X	X	X	X	X		X			
D														X	
E			X	X	X					X		X			
F					X	X	X	X				X	X		

Ordering by Added Greedy: {A,C,D,B,E,F}
Ordering by Previous Failures: {A,E,B,D,F,C}

Plotting on a Curve

Test \ Faults	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	X	X	X	X											
B					X	X	X								
C					X	X	X	X	X	X		X			
D														X	
E			X	X	X					X		X			
F					X	X	X	X				X	X		

Ordering by Added Greedy: {A,C,D,B,E,F}

Ordering by Previous Failures: {A,E,B,D,F,C}

Plotting on a Curve

Test \ Faults	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	X	X	X	X											
B					X	X	X								
C					X	X	X	X	X	X		X			
D														X	
E			X	X	X					X		X			
F					X	X	X	X				X	X		

Ordering by Added Greedy: {A,C,D,B,E,F}

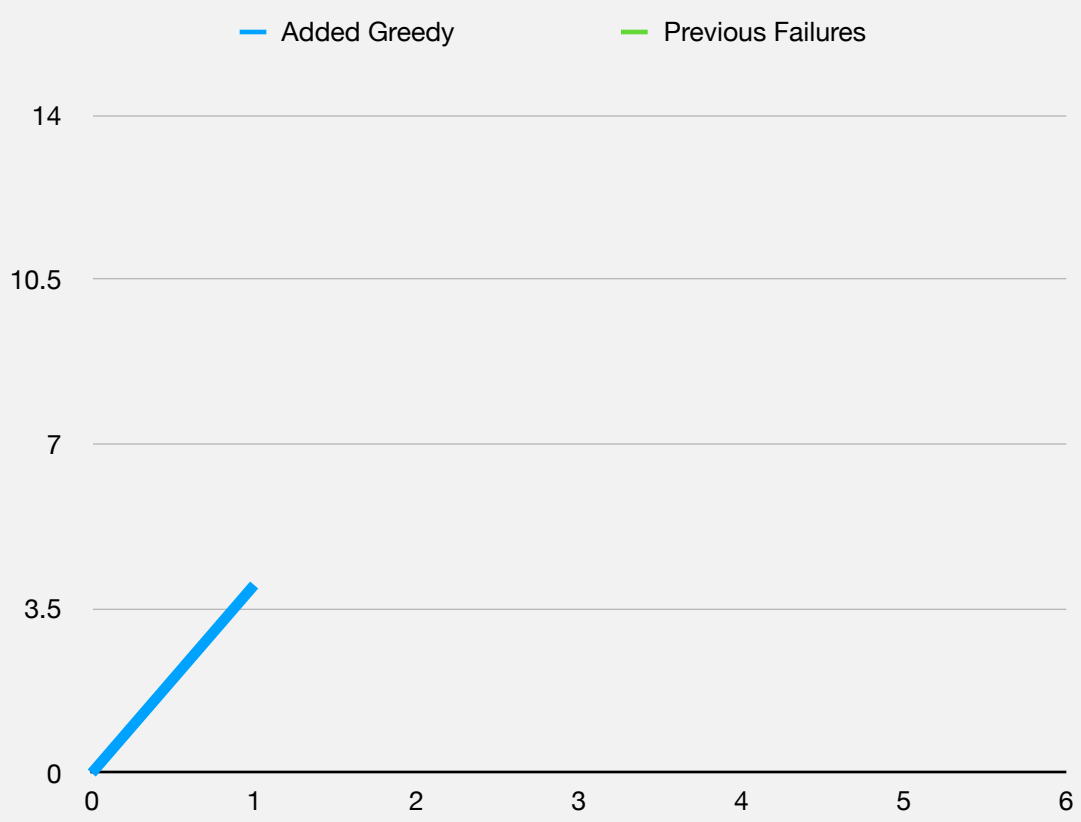
Ordering by Previous Failures: {A,E,B,D,F,C}

Plotting on a Curve

Test \ Faults	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	X	X	X	X											
B					X	X	X								
C					X	X	X	X	X	X		X			
D														X	
E			X	X	X					X		X			
F					X	X	X	X				X	X		

Ordering by Added Greedy: {A,C,D,B,E,F}

Ordering by Previous Failures: {A,E,B,D,F,C}

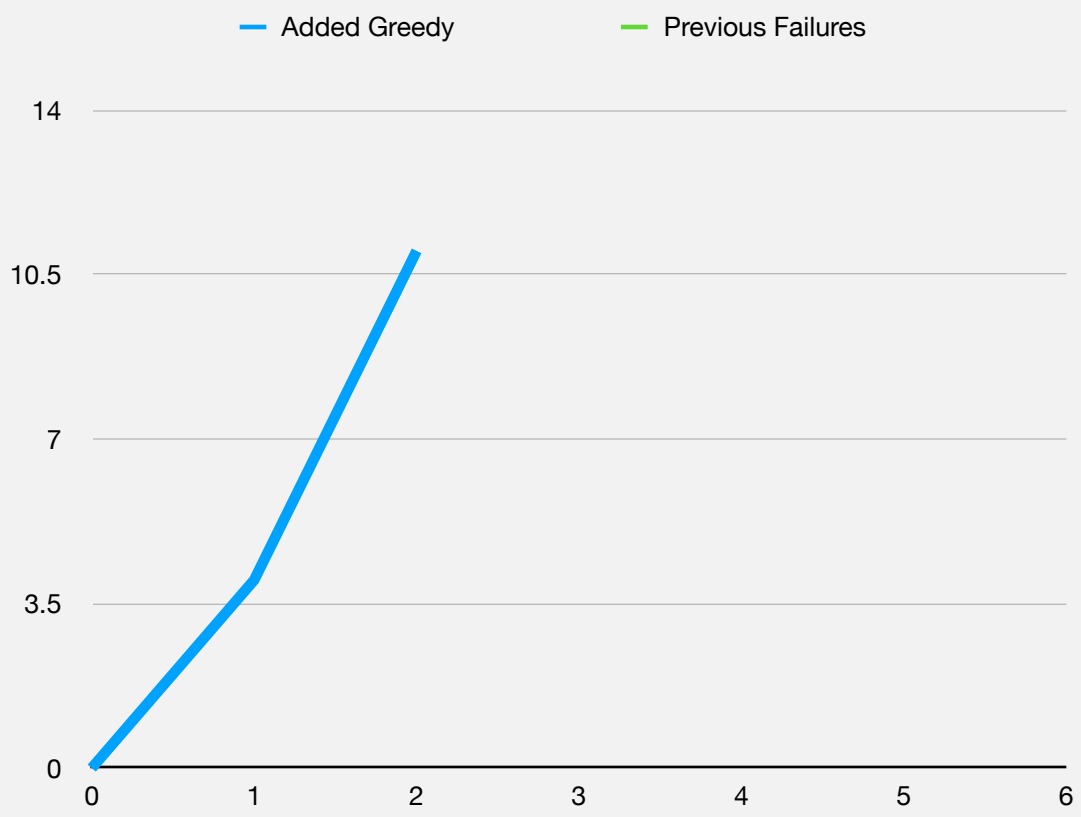


Plotting on a Curve

Test \ Faults	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	X	X	X	X											
B					X	X	X								
C					X	X	X	X	X	X		X			
D														X	
E			X	X	X					X		X			
F					X	X	X	X				X	X		

Ordering by Added Greedy: {A,C,D,B,E,F}

Ordering by Previous Failures: {A,E,B,D,F,C}

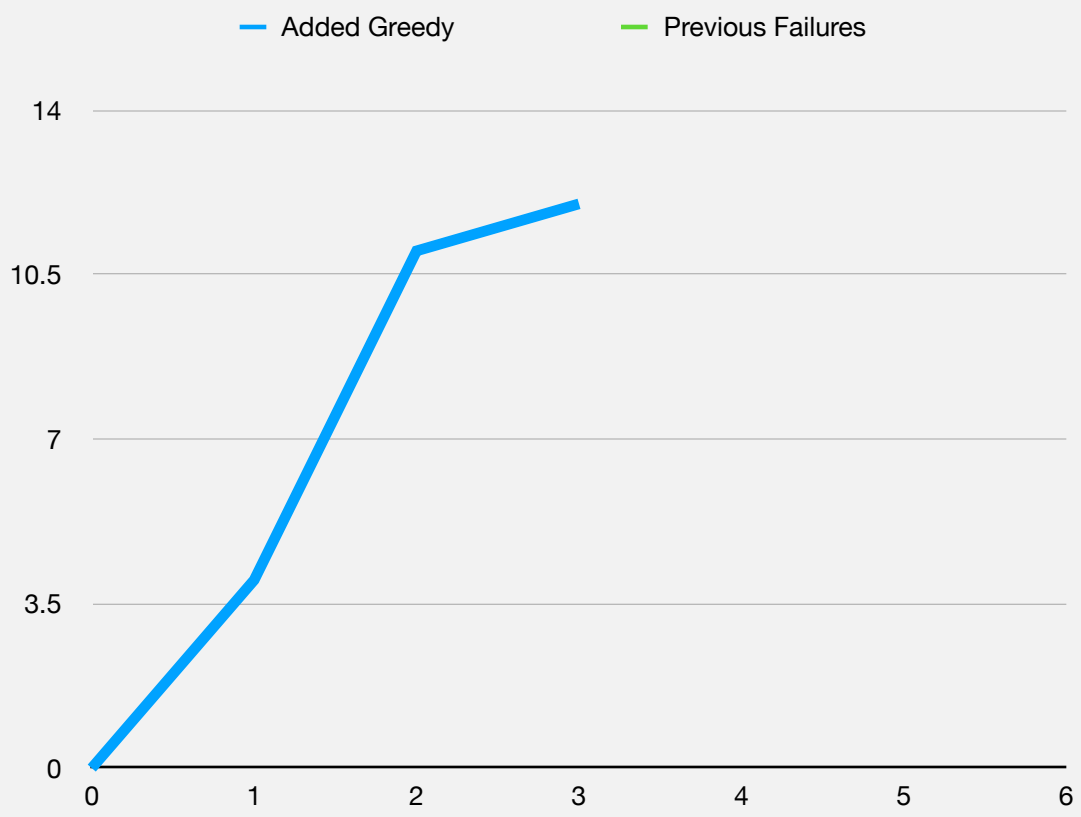


Plotting on a Curve

Test \ Faults	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	X	X	X	X											
B					X	X	X								
C					X	X	X	X	X	X		X			
D														X	
E			X	X	X					X		X			
F					X	X	X	X				X	X		

Ordering by Added Greedy: {A,C,D,B,E,F}

Ordering by Previous Failures: {A,E,B,D,F,C}

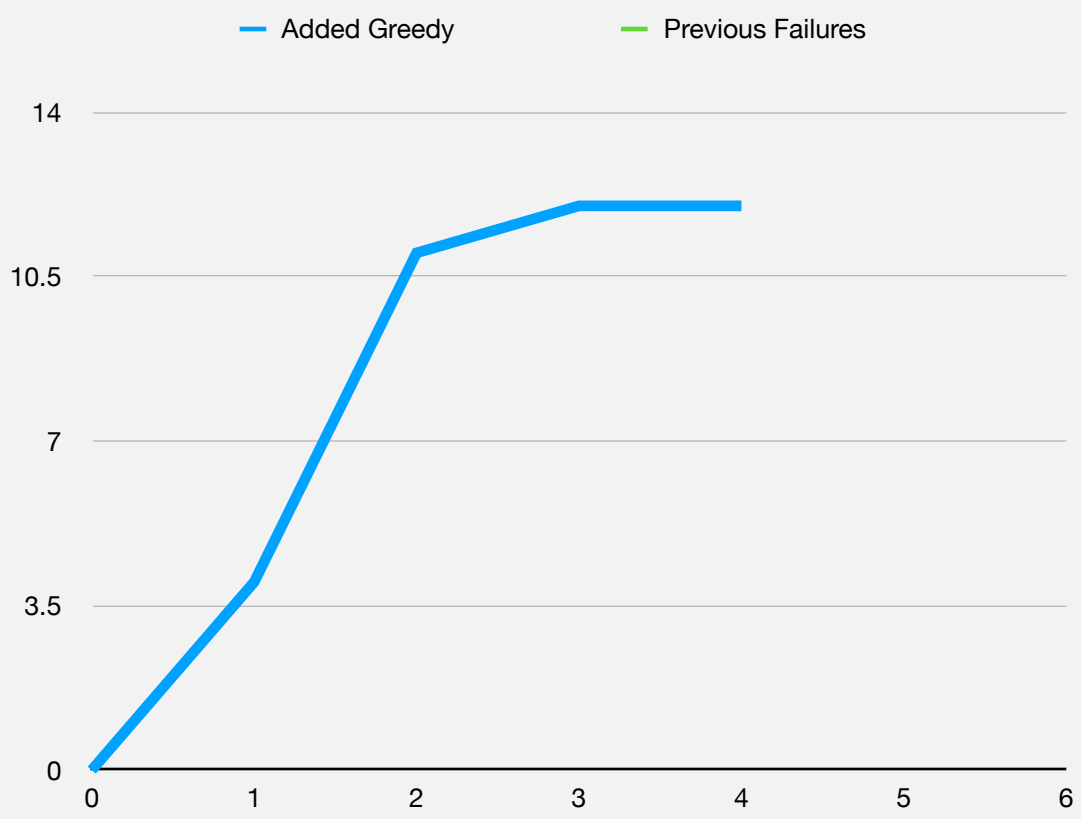


Plotting on a Curve

Test \ Faults	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	X	X	X	X											
B					X	X	X								
C					X	X	X	X	X	X		X			
D														X	
E			X	X	X					X		X			
F					X	X	X	X				X	X		

Ordering by Added Greedy: {A,C,D,B,E,F}

Ordering by Previous Failures: {A,E,B,D,F,C}

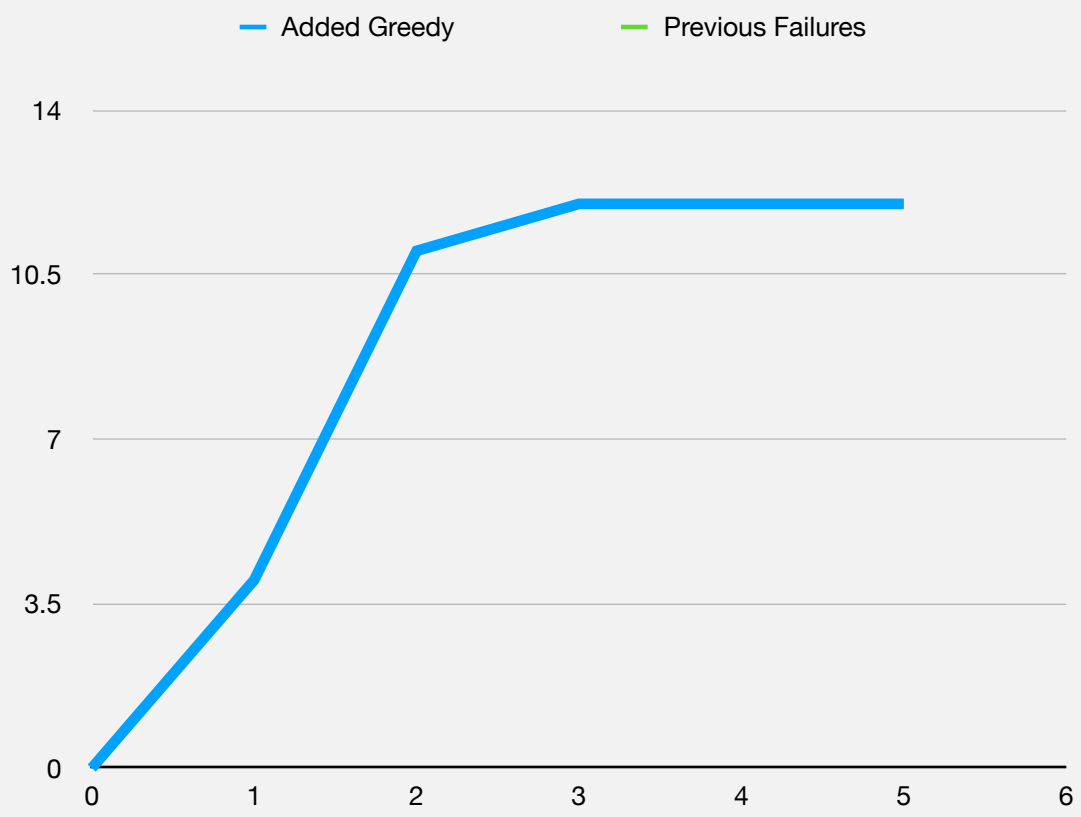


Plotting on a Curve

Test \ Faults	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	X	X	X	X											
B					X	X	X								
C					X	X	X	X	X	X		X			
D														X	
E			X	X	X					X		X			
F					X	X	X	X				X	X		

Ordering by Added Greedy: {A,C,D,B,E,F}

Ordering by Previous Failures: {A,E,B,D,F,C}

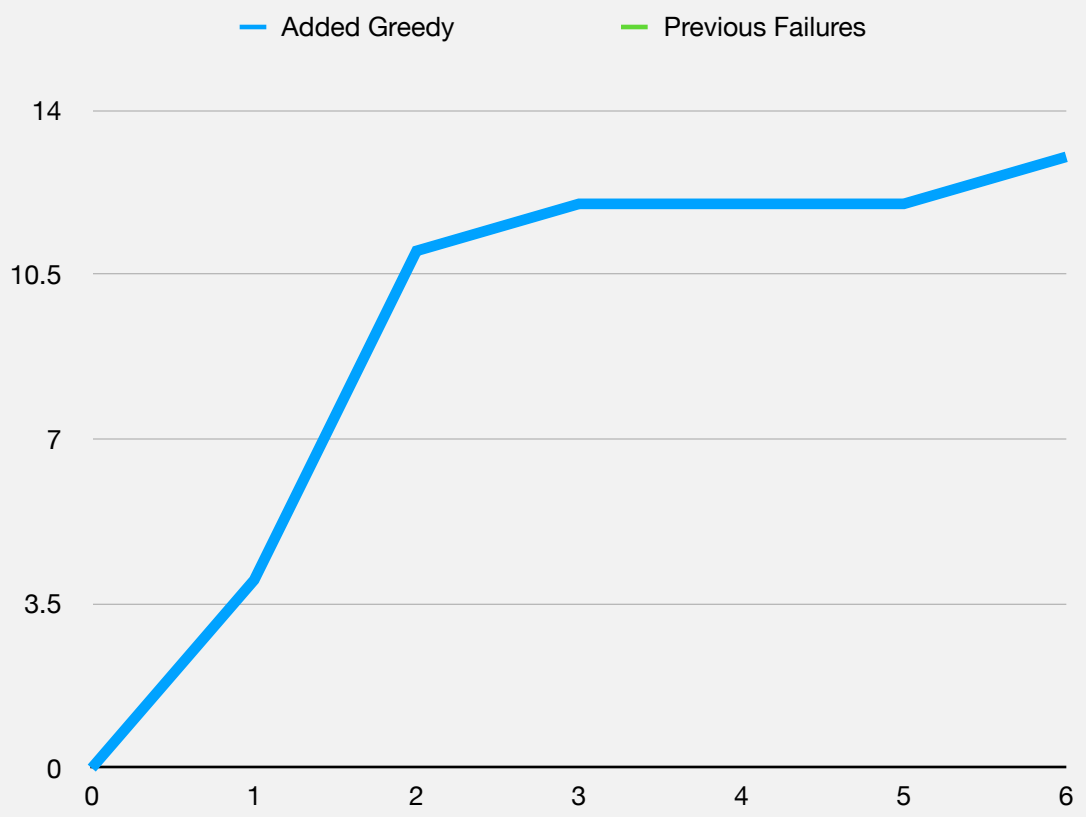


Plotting on a Curve

Test \ Faults	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	X	X	X	X											
B					X	X	X								
C					X	X	X	X	X	X		X			
D														X	
E			X	X	X					X		X			
F					X	X	X	X				X	X		

Ordering by Added Greedy: {A,C,D,B,E,F}

Ordering by Previous Failures: {A,E,B,D,F,C}

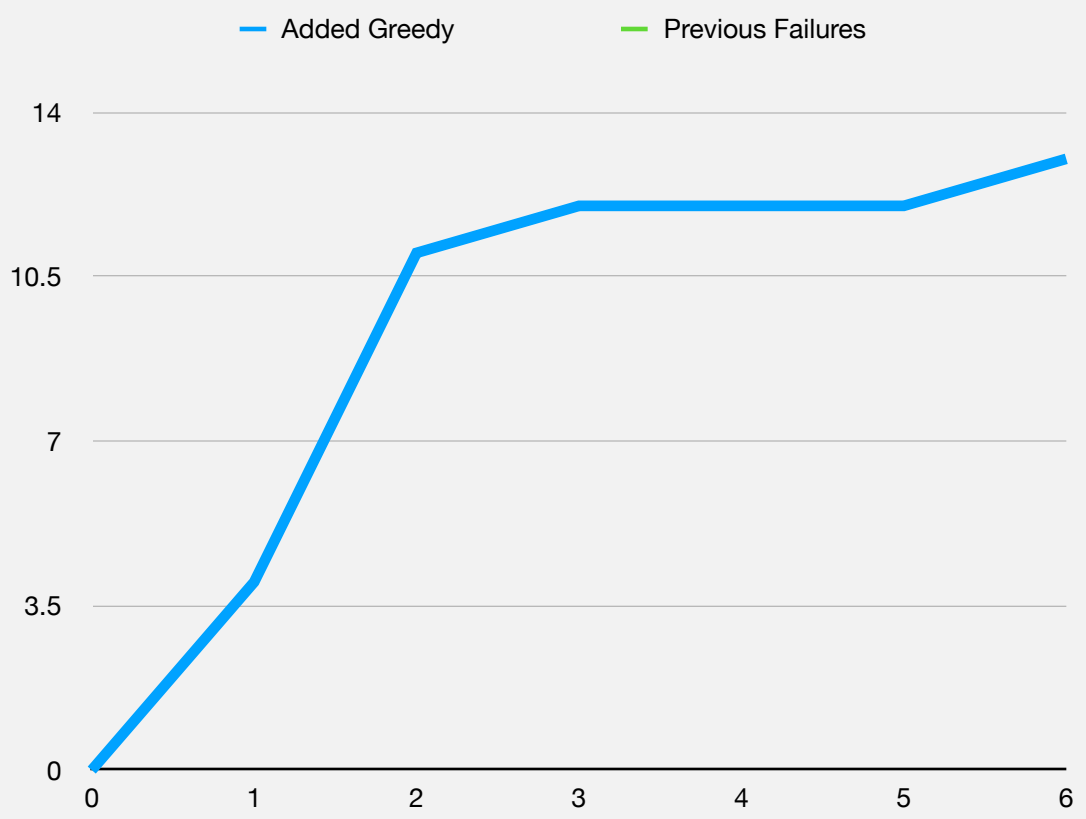


Plotting on a Curve

Test \ Faults	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	X	X	X	X											
B					X	X	X								
C					X	X	X	X	X	X		X			
D														X	
E			X	X	X					X		X			
F					X	X	X	X				X	X		

Ordering by Added Greedy: {A,C,D,B,E,F}

Ordering by Previous Failures: {A,E,B,D,F,C}

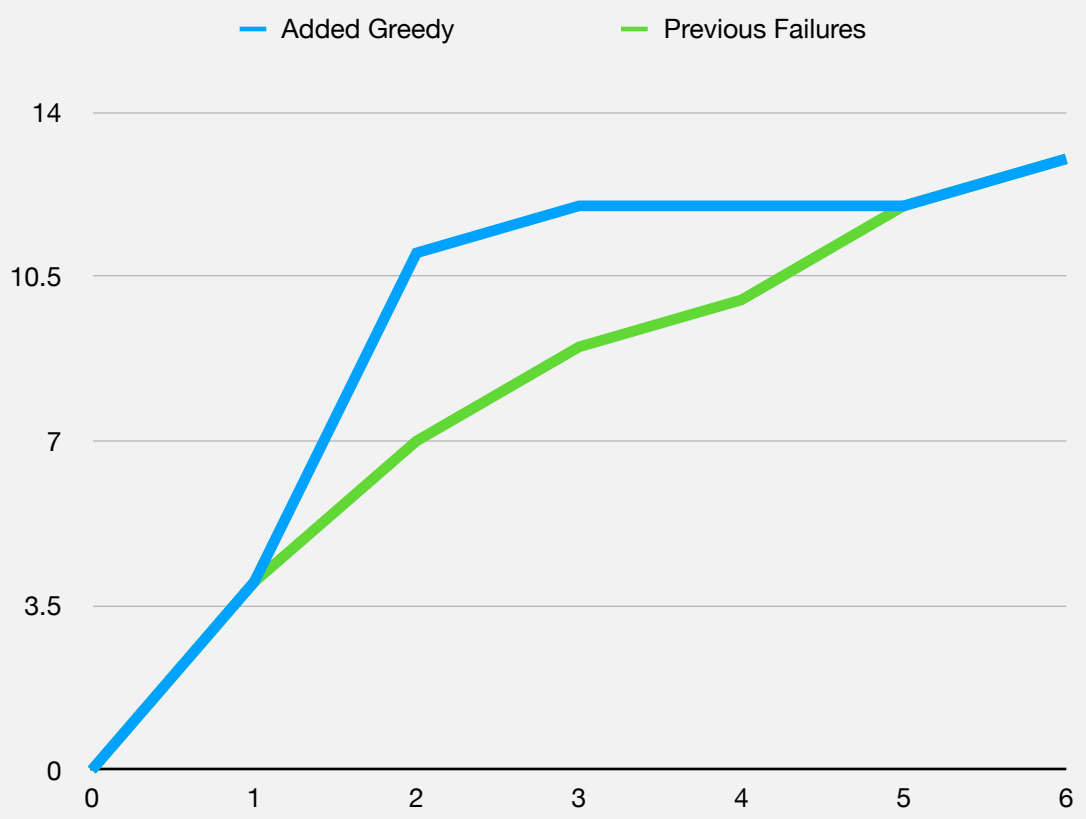


Plotting on a Curve

Test \ Faults	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	X	X	X	X											
B					X	X	X								
C					X	X	X	X	X	X		X			
D														X	
E			X	X	X					X		X			
F					X	X	X	X				X	X		

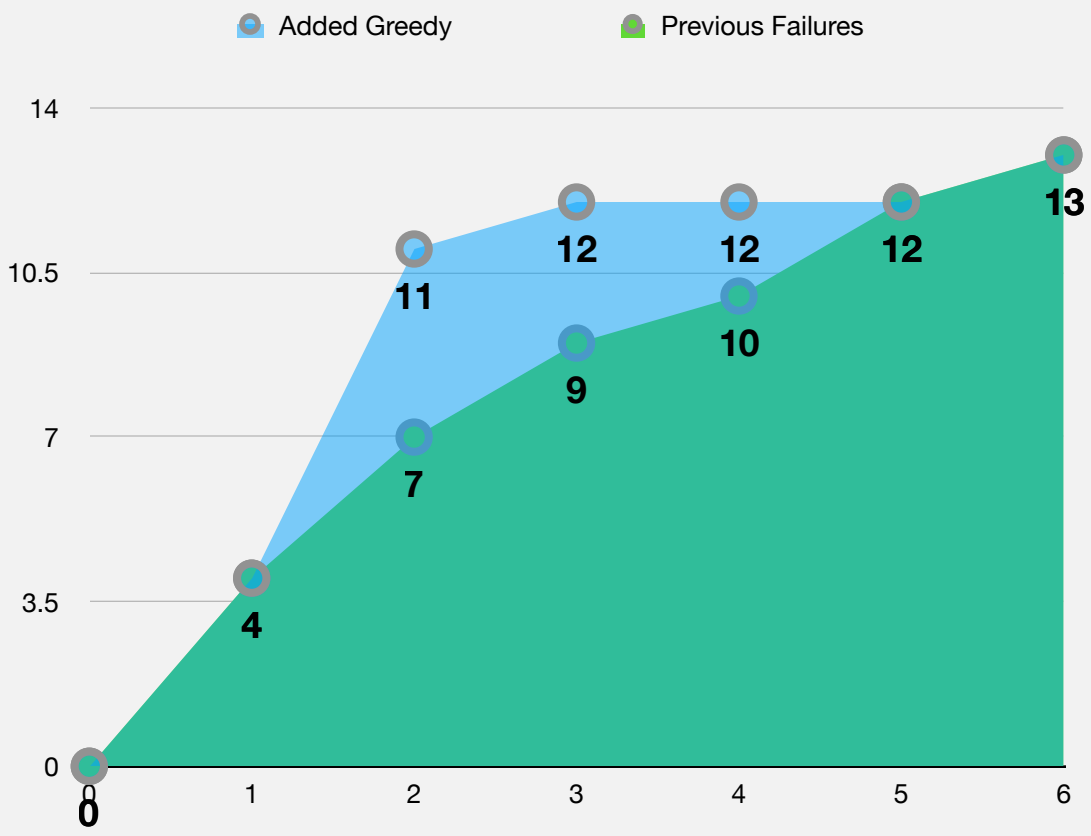
Ordering by Added Greedy: {A,C,D,B,E,F}

Ordering by Previous Failures: {A,E,B,D,F,C}



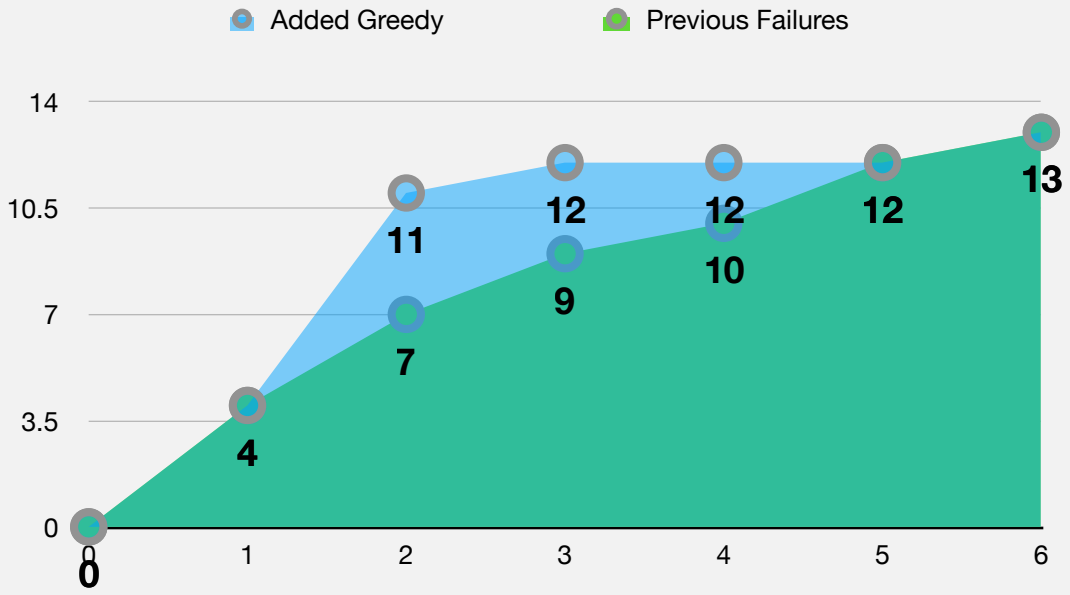
Plotting on a Curve

Test \ Faults	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	X	X	X	X											
B					X	X	X								
C					X	X	X	X	X	X		X			
D														X	
E			X	X	X					X		X			
F					X	X	X	X				X	X		



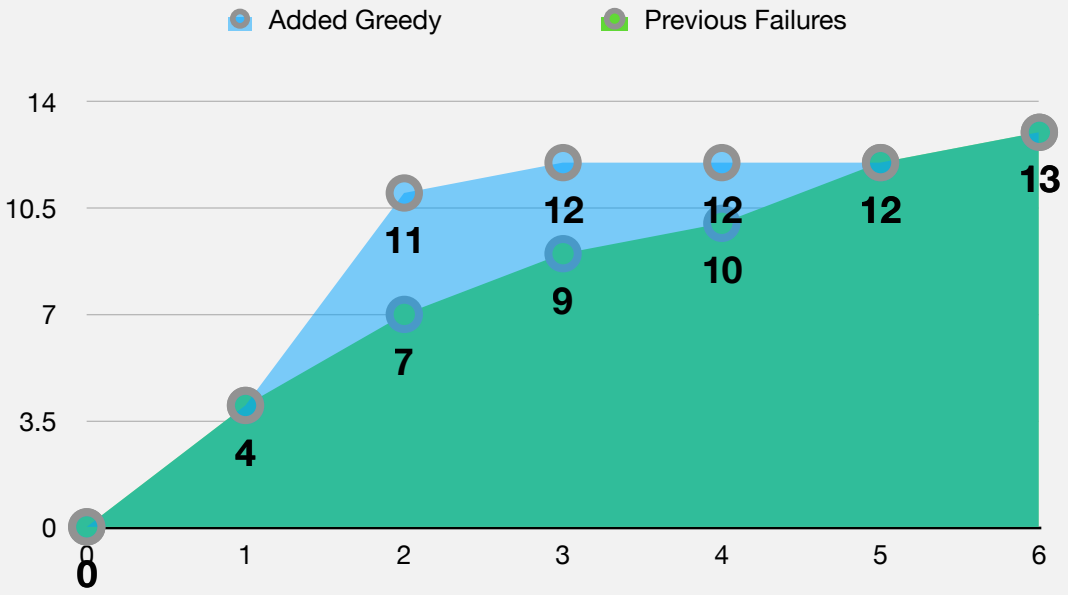
Greater area under the curve → Finding more faults early on.

APFD: Average Percentage of Faults Detected



Test \ Faults	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	X	X	X	X											
B					X	X	X								
C					X	X	X	X	X	X		X			
D														X	
E			X	X	X					X		X			
F					X	X	X	X				X	X		

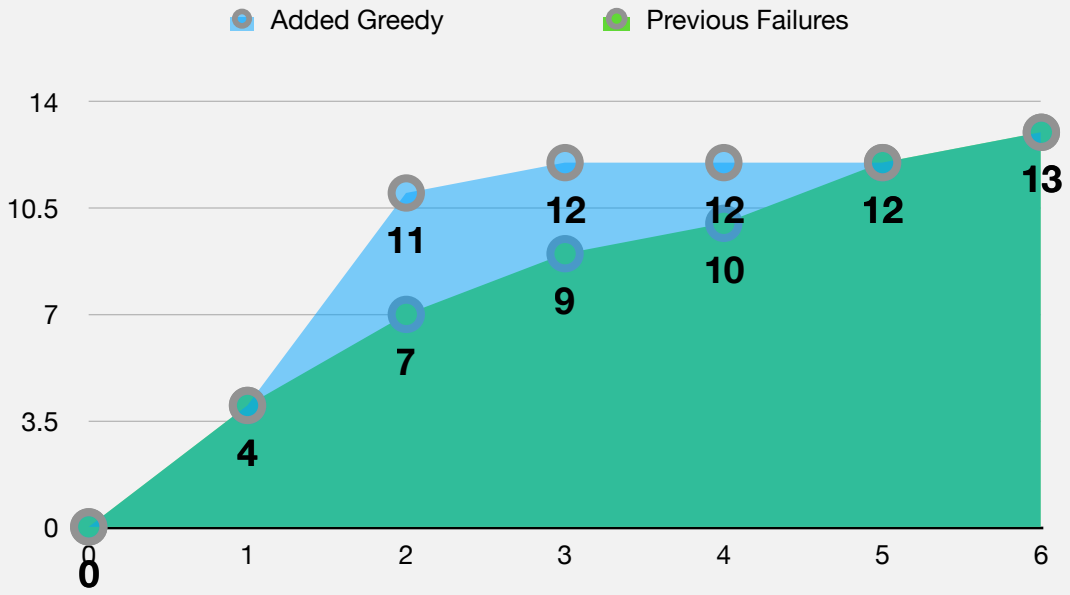
APFD: Average Percentage of Faults Detected



Test \ Faults	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	X	X	X	X											
B					X	X	X								
C					X	X	X	X	X	X		X			
D														X	
E			X	X	X					X		X			
F					X	X	X	X				X	X		

$$APFD = 1 - \frac{TF_1 + TF_2 + \ldots + TF_f}{nf} + \frac{1}{2n}$$

APFD: Average Percentage of Faults Detected



Test \ Faults	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	X	X	X	X											
B					X	X	X								
C					X	X	X	X	X	X		X			
D														X	
E			X	X	X					X		X			
F					X	X	X	X				X	X		

$$APFD = 1 - \frac{TF_1 + TF_2 + \ldots + TF_f}{nf} + \frac{1}{2n}$$

Where

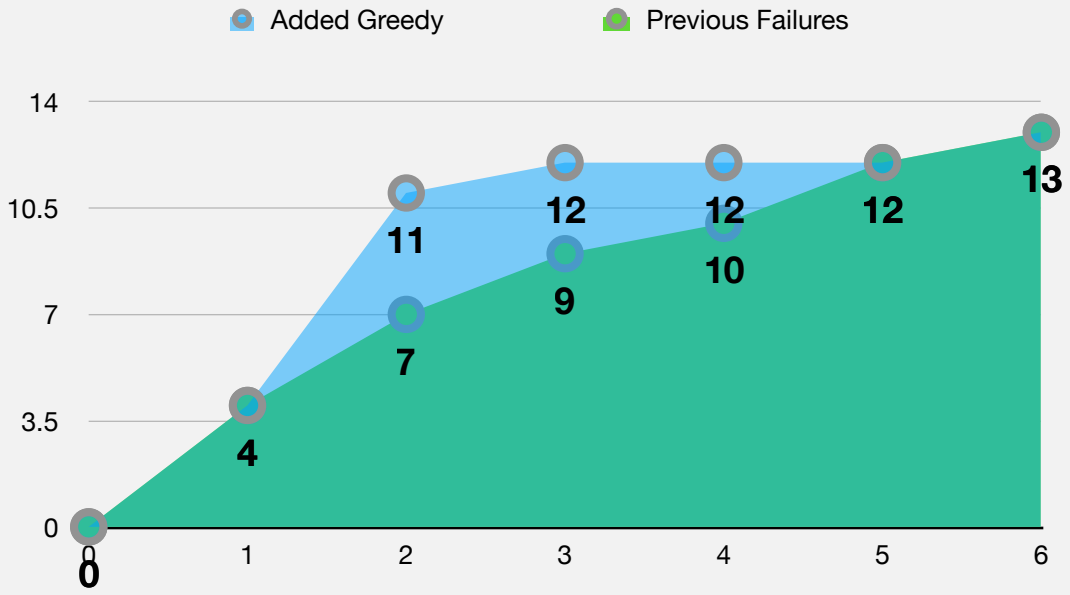
TF_i is the position of the first test that detects fault i.

n is the number of tests.

f is the number of faults.

If a fault is never detected, the value is $n + 1$

APFD: Average Percentage of Faults Detected



Test \ Faults	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	X	X	X	X											
B					X	X	X								
C					X	X	X	X	X	X		X			
D														X	
E			X	X	X					X		X			
F					X	X	X	X				X	X		

$$APFD = 1 - \frac{TF_1 + TF_2 + \ldots + TF_f}{nf} + \frac{1}{2n}$$

Where

TF_i is the position of the first test that detects fault i.

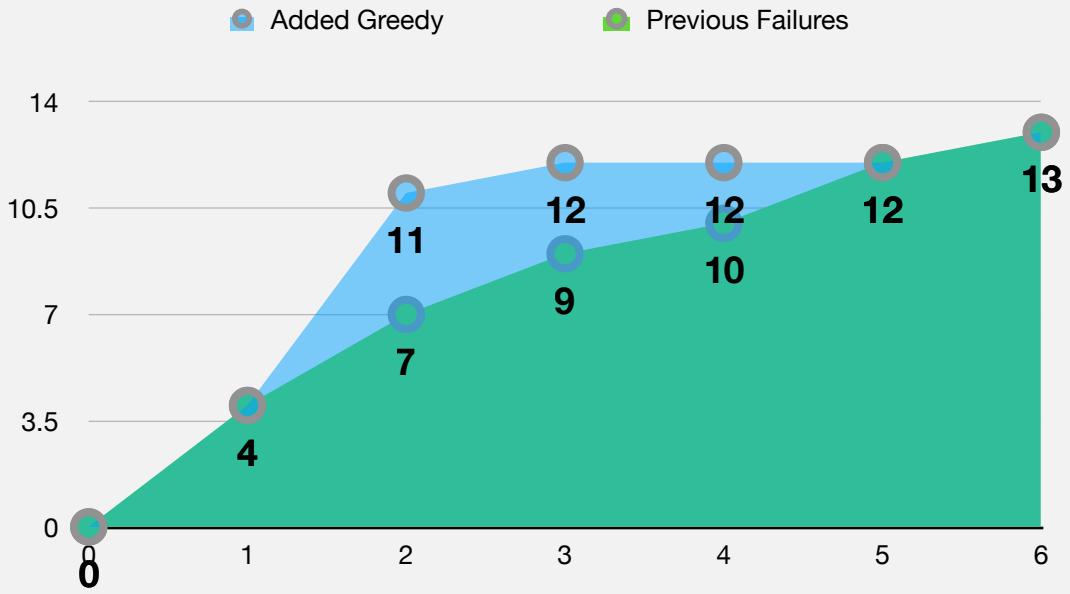
n is the number of tests.

f is the number of faults.

If a fault is never detected, the value is $n + 1$

For Average Greedy (A,C,D,B,E,F):

APFD: Average Percentage of Faults Detected



Test \ Faults	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	X	X	X	X											
B					X	X	X								
C					X	X	X	X	X	X		X			
D														X	
E			X	X	X					X		X			
F					X	X	X	X				X	X		

$$APFD = 1 - \frac{TF_1 + TF_2 + \ldots + TF_f}{nf} + \frac{1}{2n}$$

Where

TF_i is the position of the first test that detects fault i.

n is the number of tests.

f is the number of faults.

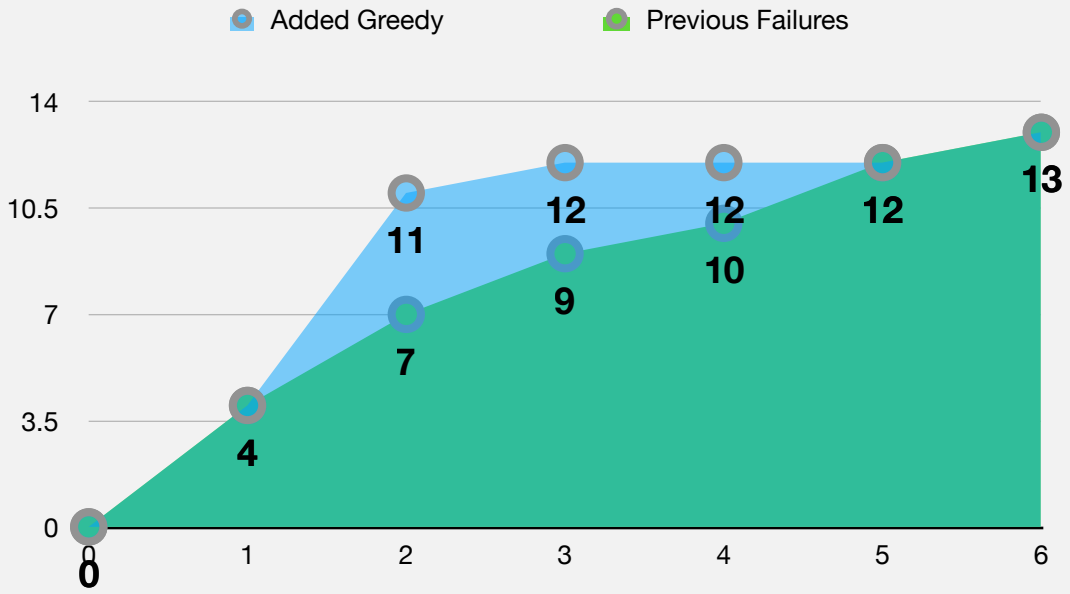
If a fault is never detected, the value is $n + 1$

For Average Greedy (A,C,D,B,E,F):

$$APFD = 1 - \frac{1+1+1+1+2+2+2+2+2+2+7+2+6+3+7}{6 \cdot 15} + \frac{1}{2 \cdot 7}$$

$$= 0.616$$

APFD: Average Percentage of Faults Detected



Test \ Faults	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	X	X	X	X											
B					X	X	X								
C					X	X	X	X	X	X		X			
D														X	
E			X	X	X					X		X			
F					X	X	X	X				X	X		

$$APFD = 1 - \frac{TF_1 + TF_2 + \ldots + TF_f}{nf} + \frac{1}{2n}$$

Where

TF_i is the position of the first test that detects fault i.

n is the number of tests.

f is the number of faults.

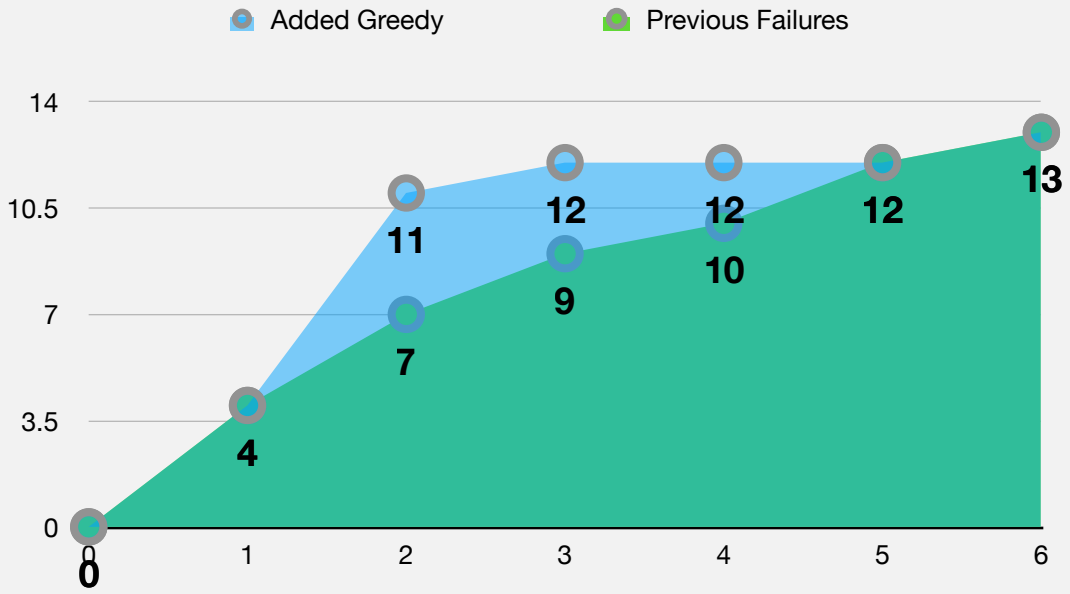
If a fault is never detected, the value is $n + 1$

For Average Greedy (A,C,D,B,E,F):

$$APFD = 1 - \frac{1+1+1+1+2+2+2+2+2+2+7+2+6+3+7}{6 \cdot 15} + \frac{1}{2 \cdot 7}$$
$$= 0.616$$

For Previous Failures (A,E,B,D,F,C):

APFD: Average Percentage of Faults Detected



Test \ Faults	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	X	X	X	X											
B					X	X	X								
C					X	X	X	X	X	X		X			
D														X	
E			X	X	X					X		X			
F					X	X	X	X				X	X		

$$APFD = 1 - \frac{TF_1 + TF_2 + \ldots + TF_f}{nf} + \frac{1}{2n}$$

Where

TF_i is the position of the first test that detects fault i.

n is the number of tests.

f is the number of faults.

If a fault is never detected, the value is $n + 1$

For Average Greedy (A,C,D,B,E,F):

$$APFD = 1 - \frac{1+1+1+1+2+2+2+2+2+2+7+2+6+3+7}{6 \cdot 15} + \frac{1}{2 \cdot 7}$$
$$= 0.616$$

For Previous Failures (A,E,B,D,F,C):

$$APFD = 1 - \frac{1+1+1+1+2+3+3+5+6+2+7+2+5+4+7}{6 \cdot 15} + \frac{1}{2 \cdot 7}$$
$$= 0.516$$

Test Selection

Rationale

Rationale

When minimising a test set you are taking a gamble.

- That the removed test-cases aren’t useful after all in detecting faults.
- Code coverage can however be misleading…

Rationale

When minimising a test set you are taking a gamble.

- That the removed test-cases aren’t useful after all in detecting faults.
- Code coverage can however be misleading…

Removed test cases have been useful in the past, and **may be useful in the future**.

Rationale

When minimising a test set you are taking a gamble.

- That the removed test-cases aren't useful after all in detecting faults.
- Code coverage can however be misleading…

Removed test cases have been useful in the past, and **may be useful in the future**.

Selection is an alternative to minimisation.

- It's ok to keep large test-suites in the code base…
- … as long as we carefully decide upon which test cases to run.
- **Base our selection on which tests cover the changed code**.

Intra-procedural Test Selection

Start with $T' = \emptyset$.

Build P and P', the CFGs corresponding to the unmodified and modified code respectively.

For $i \in 1, \ldots, |T|$:

$edgeList_{T_i} = traceTest(P, t_i)$

$difference = compare(edgeList_{t_i}, P')$

If $difference$

$T' = T' \cup t_i$

Intra-procedural Test Selection

Start with $T' = \emptyset$.

Build P and P', the CFGs corresponding to the unmodified and modified code respectively.

For $i \in 1, \ldots, |T|$:

$edgeList_{T_i} = traceTest(P, t_i)$

$difference = compare(edgeList_{t_i}, P')$

Walk through the list of edges in P'. If the labels for either nodes are different, return True.

If $difference$

$T' = T' \cup t_i$

Intra-procedural Test Selection

Start with $T' = \emptyset$.

Build P and P', the CFGs corresponding to the unmodified and modified code respectively.

For $i \in 1, \ldots, |T|$:

$edgeList_{T_i} = traceTest(P, t_i)$

$difference = compare(edgeList_{t_i}, P')$

Walk through the list of edges in P'. If the labels for either nodes are different, return True.

If $difference$

$T' = T' \cup t_i$

Add tests to T' only if the covered code has changed.

Test Selection example

```
public static int avg(File inputFile) throws Exception {
 1   int count = 0; int sum = 0;
 2   Scanner in = new Scanner(new FileReader(inputFile));
 3   while(in.hasNext()) {
 4       String line = in.next();
 5       if (! StringUtils.isNumeric(line)) {
 6           throw new NumberFormatException();
 7       } else {
 8           sum += Integer.parseInt(line);
 9           count++;
10       }
11   }
12   in.close();
13   return count > 0 ? sum / count : 0;
 }
```

```
@Test
public void t1() throws Exception {
    assertEquals(0, TestSelection.avg(empty));
}

@Test
public void t2() throws Exception {
    assertThrows(NumberFormatException.class, () -> {
        TestSelection.avg(nonNumeric);
    });
}

@Test
public void t3() throws Exception {
    assertEquals(2, TestSelection.avg(numbers123));
}
```

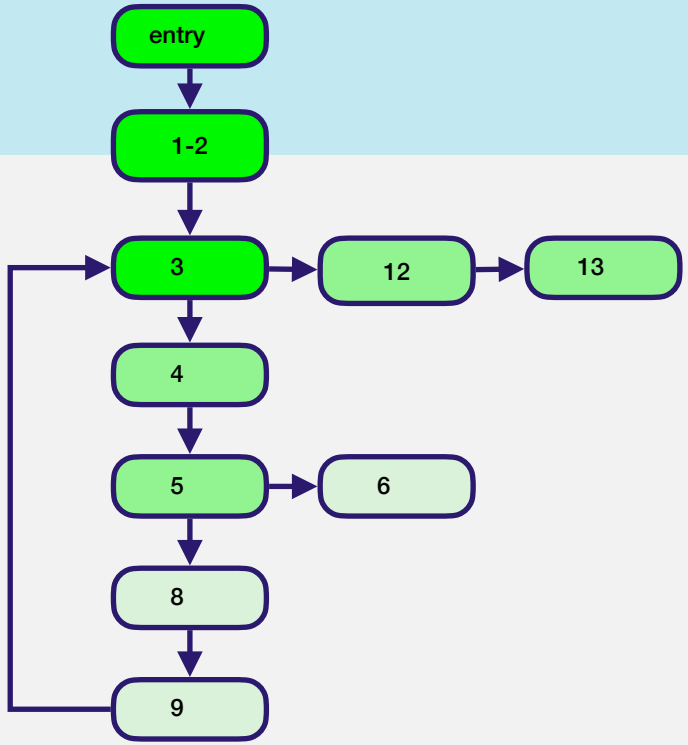
Test Selection example

```
public static int avg(File inputFile) throws Exception {
 1   int count = 0; int sum = 0;
 2   Scanner in = new Scanner(new FileReader(inputFile));
 3   while(in.hasNext()) {
 4       String line = in.next();
 5       if (! StringUtils.isNumeric(line)) {
 6           throw new NumberFormatException();
 7       } else {
 8           sum += Integer.parseInt(line);
 9           count++;
10       }
11   }
12   in.close();
13   return count > 0 ? sum / count : 0;
 }
```

```
@Test
public void t1() throws Exception {
    assertEquals(0, TestSelection.avg(empty));
}

@Test
public void t2() throws Exception {
    assertThrows(NumberFormatException.class, () -> {
        TestSelection.avg(nonNumeric);
    });
}

@Test
public void t3() throws Exception {
    assertEquals(2, TestSelection.avg(numbers123));
}
```



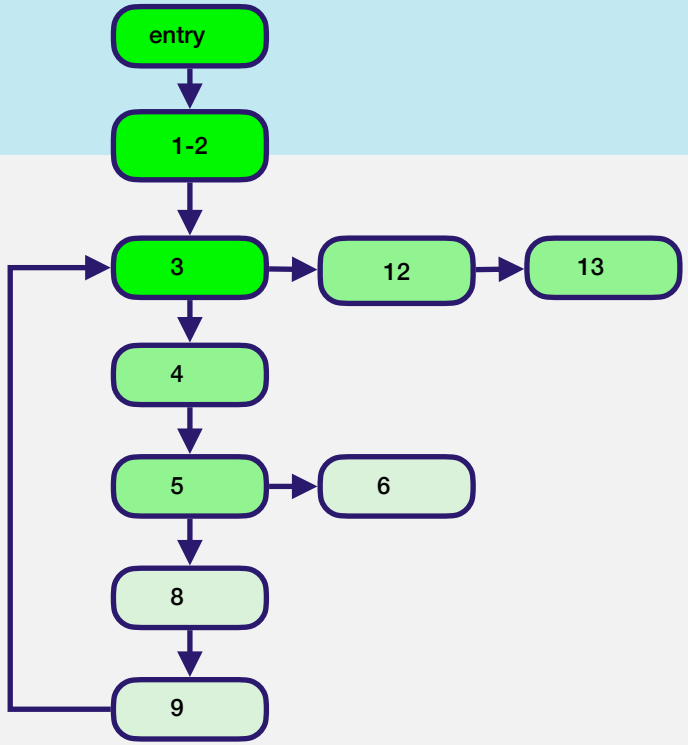
Test Selection example

```
public static int avg(File inputFile) throws Exception {
 1   int count = 0; int sum = 0;
 2   Scanner in = new Scanner(new FileReader(inputFile));
 3   while(in.hasNext()) {
 4       String line = in.next();
 5       if (! StringUtils.isNumeric(line)) {
 6           throw new NumberFormatException();
 7       } else {
 8           sum += Integer.parseInt(line);
 9           count++;
10       }
11   }
12   in.close();
13   return count > 0 ? sum / count : 0;
 }
```

```
@Test
public void t1() throws Exception {
    assertEquals(0, TestSelection.avg(empty));
}

@Test
public void t2() throws Exception {
    assertThrows(NumberFormatException.class, () -> {
        TestSelection.avg(nonNumeric);
    });
}

@Test
public void t3() throws Exception {
    assertEquals(2, TestSelection.avg(numbers123));
}
```



Test	Edges Traversed

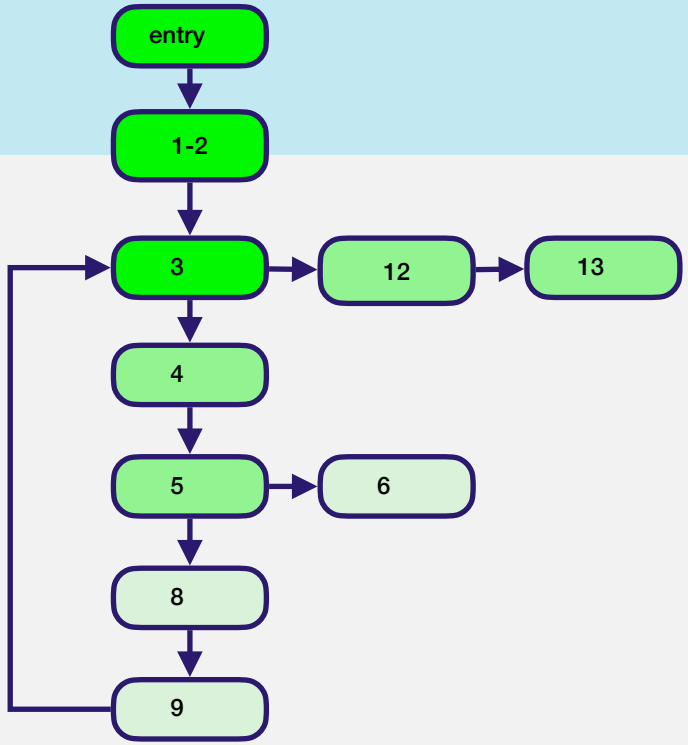
Test Selection example

```
public static int avg(File inputFile) throws Exception {
 1   int count = 0; int sum = 0;
 2   Scanner in = new Scanner(new FileReader(inputFile));
 3   while(in.hasNext()) {
 4       String line = in.next();
 5       if (! StringUtils.isNumeric(line)) {
 6           throw new NumberFormatException();
 7       } else {
 8           sum += Integer.parseInt(line);
 9           count++;
10       }
11   }
12   in.close();
13   return count > 0 ? sum / count : 0;
 }
```

```
@Test
public void t1() throws Exception {
    assertEquals(0, TestSelection.avg(empty));
}

@Test
public void t2() throws Exception {
    assertThrows(NumberFormatException.class, () -> {
        TestSelection.avg(nonNumeric);
    });
}

@Test
public void t3() throws Exception {
    assertEquals(2, TestSelection.avg(numbers123));
}
```



Test	Edges Traversed
t1	(entry, 1-2), (1-2, 3), (3, 12), (12, 13)

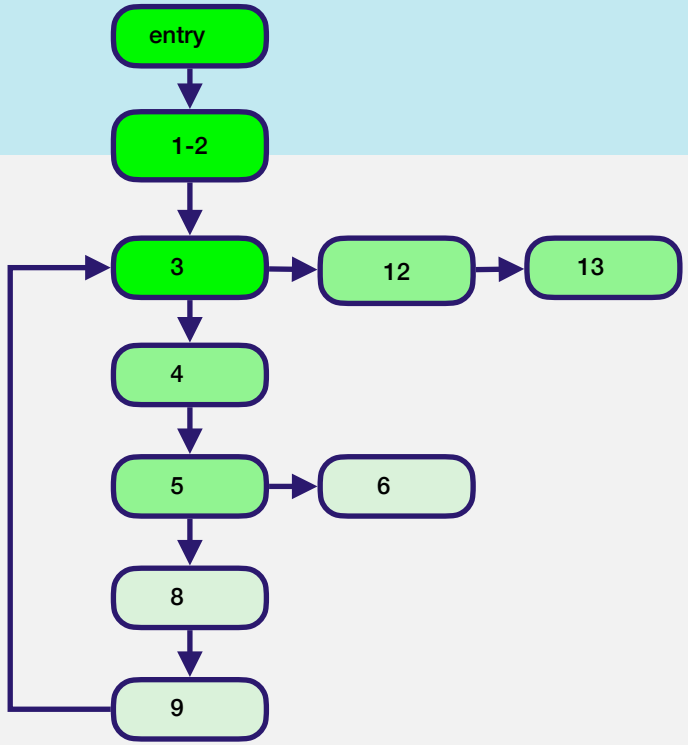
Test Selection example

```
public static int avg(File inputFile) throws Exception {
 1   int count = 0; int sum = 0;
 2   Scanner in = new Scanner(new FileReader(inputFile));
 3   while(in.hasNext()) {
 4       String line = in.next();
 5       if (! StringUtils.isNumeric(line)) {
 6           throw new NumberFormatException();
 7       } else {
 8           sum += Integer.parseInt(line);
 9           count++;
10       }
11   }
12   in.close();
13   return count > 0 ? sum / count : 0;
}
```

```
@Test
public void t1() throws Exception {
    assertEquals(0, TestSelection.avg(empty));
}

@Test
public void t2() throws Exception {
    assertThrows(NumberFormatException.class, () -> {
        TestSelection.avg(nonNumeric);
    });
}

@Test
public void t3() throws Exception {
    assertEquals(2, TestSelection.avg(numbers123));
}
```



Test	Edges Traversed
t1	(entry, 1-2), (1-2, 3), (3, 12), (12, 13)
t2	(entry, 1-2), (1-2, 3), (3, 4), (4, 5), (5, 6)

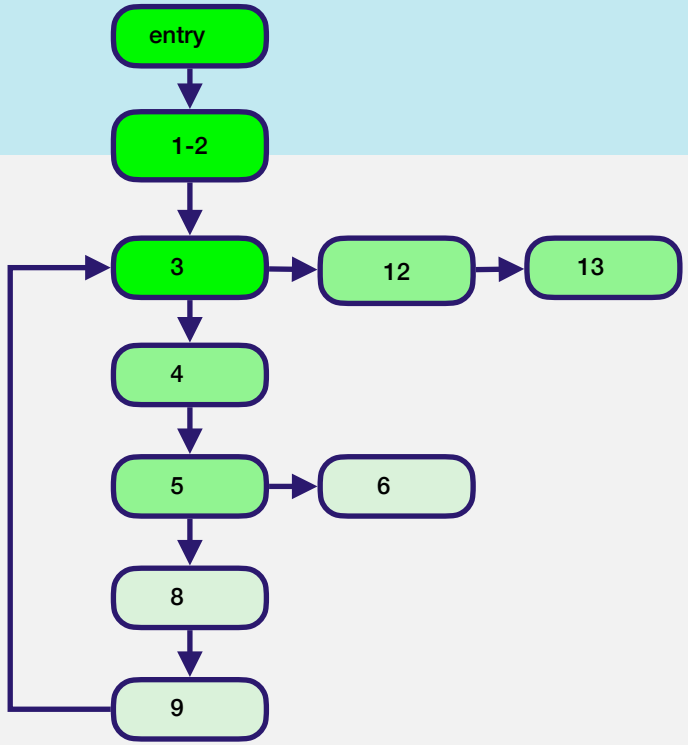
Test Selection example

```
public static int avg(File inputFile) throws Exception {
 1   int count = 0; int sum = 0;
 2   Scanner in = new Scanner(new FileReader(inputFile));
 3   while(in.hasNext()) {
 4       String line = in.next();
 5       if (! StringUtils.isNumeric(line)) {
 6           throw new NumberFormatException();
 7       } else {
 8           sum += Integer.parseInt(line);
 9           count++;
10       }
11   }
12   in.close();
13   return count > 0 ? sum / count : 0;
}
```

```
@Test
public void t1() throws Exception {
    assertEquals(0, TestSelection.avg(empty));
}

@Test
public void t2() throws Exception {
    assertThrows(NumberFormatException.class, () -> {
        TestSelection.avg(nonNumeric);
    });
}

@Test
public void t3() throws Exception {
    assertEquals(2, TestSelection.avg(numbers123));
}
```



Test	Edges Traversed
t1	(entry, 1-2), (1-2, 3), (3, 12), (12, 13)
t2	(entry, 1-2), (1-2, 3), (3, 4), (4, 5), (5, 6)
t3	(entry, 1-2), (1-2, 3), (3, 4), (4, 5), (5, 8), (8, 9), (9, 3), (3, 12), (12, 13)

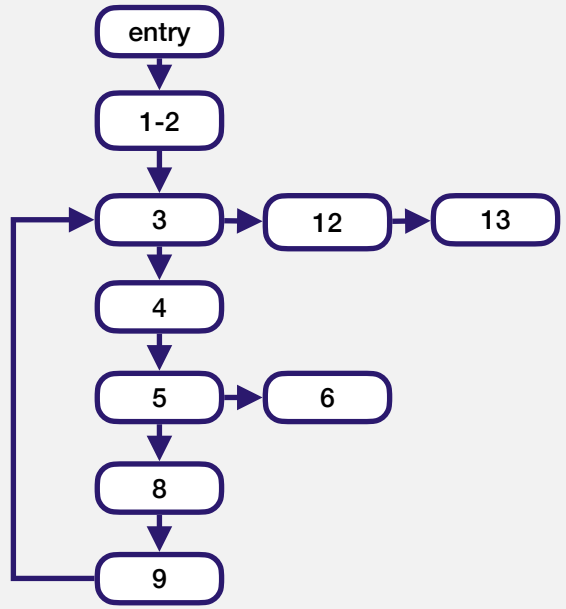
Test Selection example

```
public static int avg(File inputFile) throws Exception {
 1   int count = 0; int sum = 0;
 2   Scanner in = new Scanner(new FileReader(inputFile));
 3   while(in.hasNext()) {
 4       String line = in.next();
 5       if (! StringUtils.isNumeric(line)) {
 6           throw new NumberFormatException();
 7       } else {
 8           sum += Integer.parseInt(line);
 9           count++;
10       }
11   }
12   in.close();
13   return count > 0 ? sum / count : 0;
}
```

```
public static int avg(File inputFile) throws Exception {
 1   int count = 0; int sum = 0;
 2   Scanner in = new Scanner(new FileReader(inputFile));
 3   while(in.hasNext()) {
 4       String line = in.next();
 5       if (! StringUtils.isNumeric(line)) {
 6         System.err.println("Bad input");
 7         throw new NumberFormatException();
 8       } else {
 9           sum += Integer.parseInt(line);
10       }
11   }
12   in.close();
13   return count > 0 ? sum / count : 0;
}
```

Test Selection example

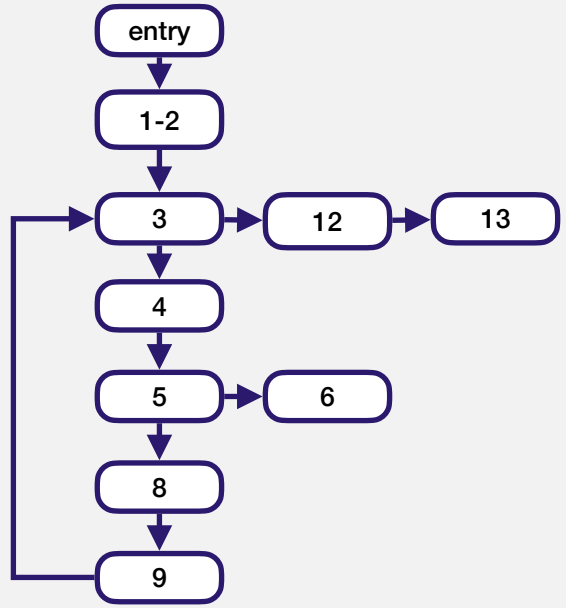
```
public static int avg(File inputFile) throws Exception {
 1   int count = 0; int sum = 0;
 2   Scanner in = new Scanner(new FileReader(inputFile));
 3   while(in.hasNext()) {
 4       String line = in.next();
 5       if (! StringUtils.isNumeric(line)) {
 6           throw new NumberFormatException();
 7       } else {
 8           sum += Integer.parseInt(line);
 9           count++;
10       }
11   }
12   in.close();
13   return count > 0 ? sum / count : 0;
}
```



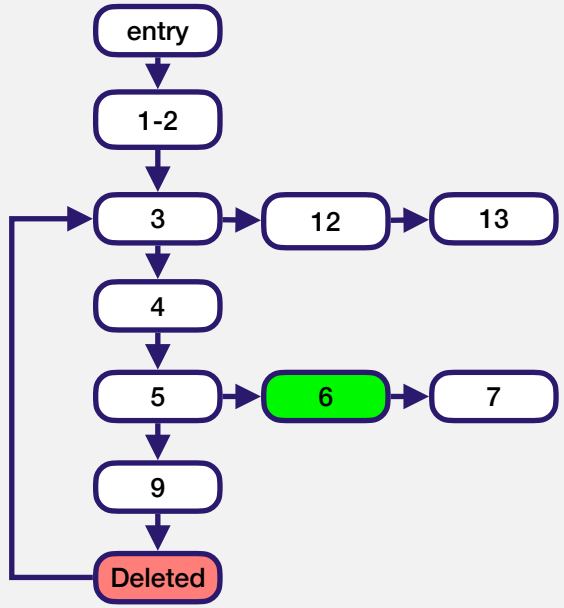

```
public static int avg(File inputFile) throws Exception {
 1   int count = 0; int sum = 0;
 2   Scanner in = new Scanner(new FileReader(inputFile));
 3   while(in.hasNext()) {
 4       String line = in.next();
 5       if (! StringUtils.isNumeric(line)) {
 6         System.err.println("Bad input");
 7         throw new NumberFormatException();
 8       } else {
 9           sum += Integer.parseInt(line);
10       }
11   }
12   in.close();
13   return count > 0 ? sum / count : 0;
}
```

Test Selection example

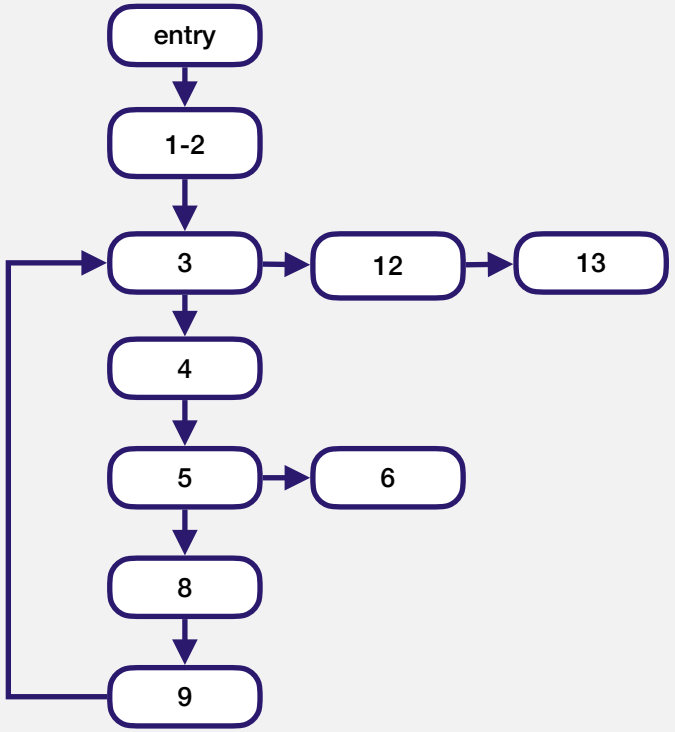
```
public static int avg(File inputFile) throws Exception {
 1   int count = 0; int sum = 0;
 2   Scanner in = new Scanner(new FileReader(inputFile));
 3   while(in.hasNext()) {
 4       String line = in.next();
 5       if (! StringUtils.isNumeric(line)) {
 6           throw new NumberFormatException();
 7       } else {
 8           sum += Integer.parseInt(line);
 9           count++;
10       }
11   }
12   in.close();
13   return count > 0 ? sum / count : 0;
}
```

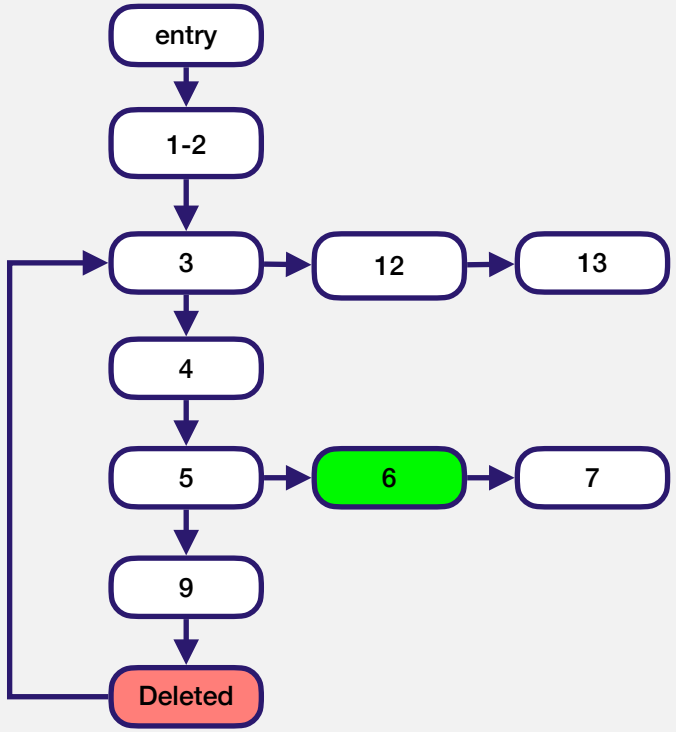



```
public static int avg(File inputFile) throws Exception {
 1   int count = 0; int sum = 0;
 2   Scanner in = new Scanner(new FileReader(inputFile));
 3   while(in.hasNext()) {
 4       String line = in.next();
 5       if (! StringUtils.isNumeric(line)) {
 6         System.err.println("Bad input");
 7         throw new NumberFormatException();
 8       } else {
 9           sum += Integer.parseInt(line);
10       }
11   }
12   in.close();
13   return count > 0 ? sum / count : 0;
}
```



Test Selection example

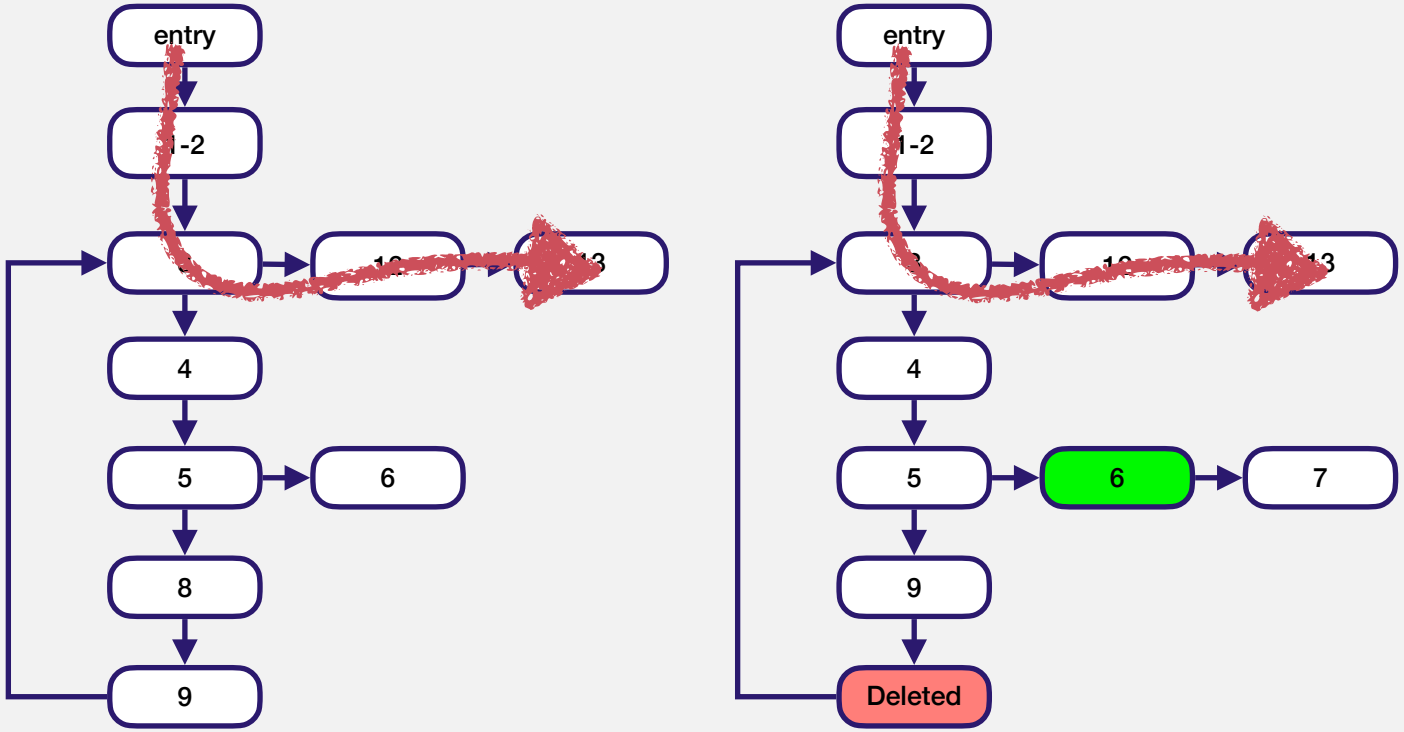




$$T' = \emptyset$$

Test	Edges Traversed
t1	(entry, 1-2), (1-2, 3), (3, 12), (12, 13)
t2	(entry, 1-2), (1-2, 3), (3, 4), (4, 5), (5, 6)
t3	(entry, 1-2), (1-2, 3), (3, 4), (4, 5), (5, 8), (8, 9), (9, 3), (3, 12), (12, 13)

Test Selection example



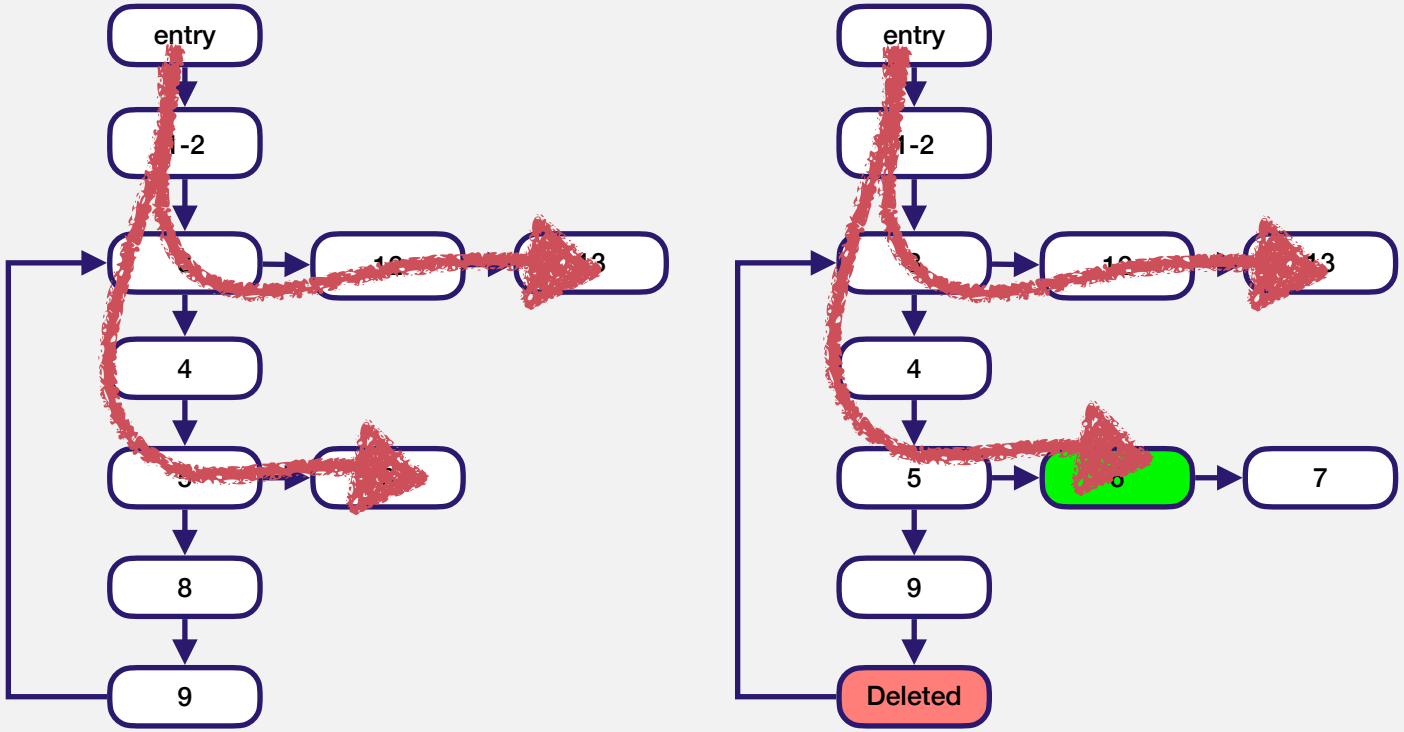
$T' = \emptyset$

Traverse t_1 in P and P'

No difference.

Test	Edges Traversed
t1	(entry, 1-2), (1-2, 3), (3, 12), (12, 13)
t2	(entry, 1-2), (1-2, 3), (3, 4), (4, 5), (5, 6)
t3	(entry, 1-2), (1-2, 3), (3, 4), (4, 5), (5, 8), (8, 9), (9, 3), (3, 12), (12, 13)

Test Selection example



$T' = \emptyset$

Traverse t_1 in P and P'

No difference.

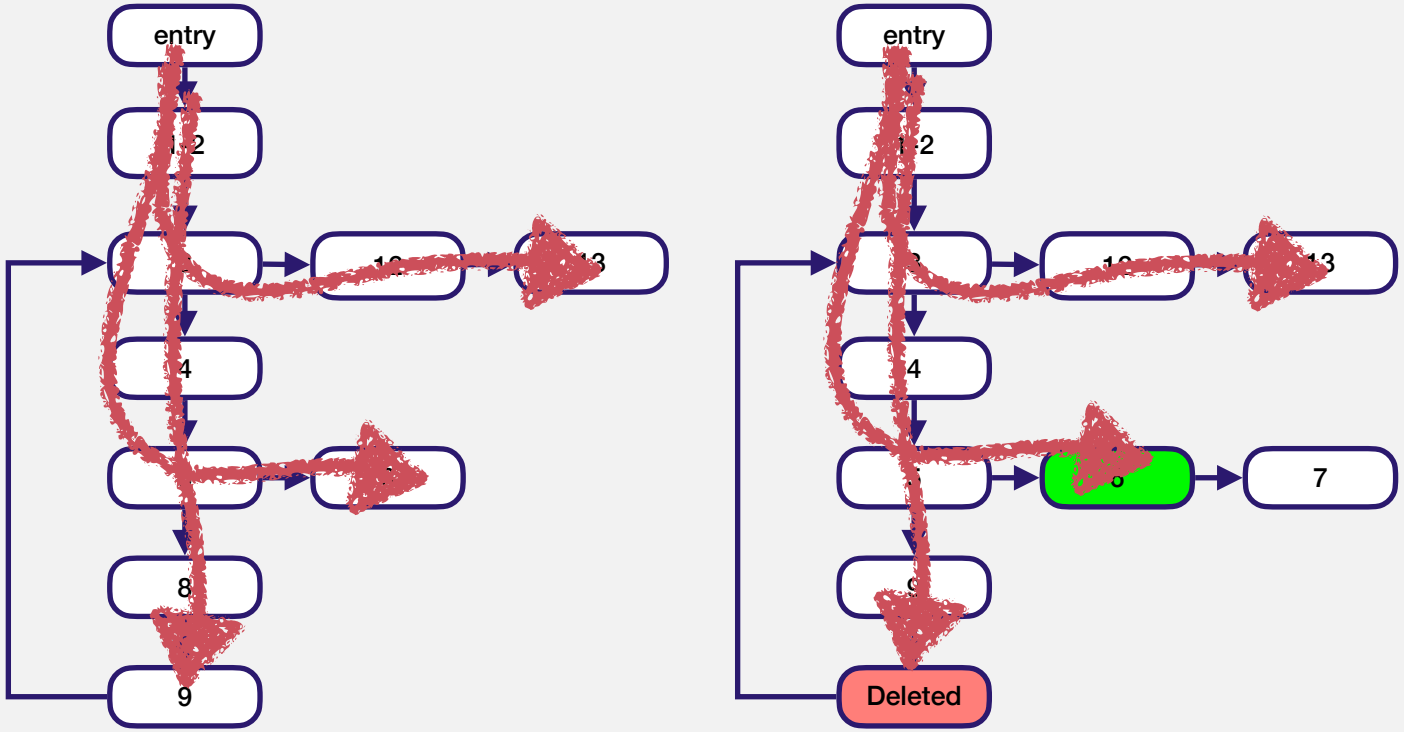
Traverse t_2 in P and P'

Difference in node 6

$T' = T' \cup \{t_2\}$

Test	Edges Traversed
t1	(entry, 1-2), (1-2, 3), (3, 12), (12, 13)
t2	(entry, 1-2), (1-2, 3), (3, 4), (4, 5), (5, 6)
t3	(entry, 1-2), (1-2, 3), (3, 4), (4, 5), (5, 8), (8, 9), (9, 3), (3, 12), (12, 13)

Test Selection example



Test	Edges Traversed
t1	(entry, 1-2), (1-2, 3), (3, 12), (12, 13)
t2	(entry, 1-2), (1-2, 3), (3, 4), (4, 5), (5, 6)
t3	(entry, 1-2), (1-2, 3), (3, 4), (4, 5), (5, 8), (8, 9), (9, 3), (3, 12), (12, 13)

$T' = \emptyset$

Traverse t_1 in P and P'

No difference.

Traverse t_2 in P and P'

Difference in node 6

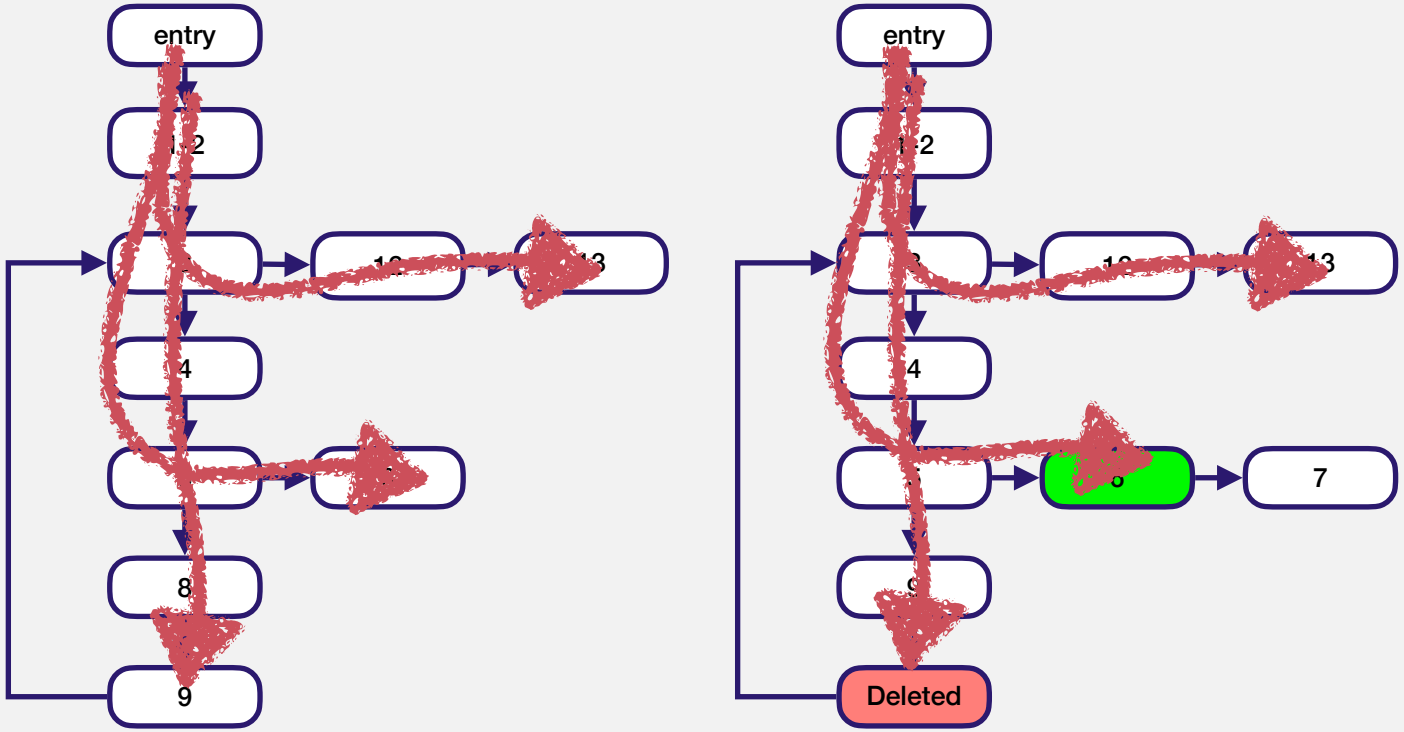
$T' = T' \cup \{t_2\}$

Traverse t_3 in P and P'

Difference in node 9

$T' = T' \cup \{t_3\}$

Test Selection example



Test	Edges Traversed
t1	(entry, 1-2), (1-2, 3), (3, 12), (12, 13)
t2	(entry, 1-2), (1-2, 3), (3, 4), (4, 5), (5, 6)
t3	(entry, 1-2), (1-2, 3), (3, 4), (4, 5), (5, 8), (8, 9), (9, 3), (3, 12), (12, 13)

$T' = \emptyset$

Traverse t_1 in P and P'

No difference.

Traverse t_2 in P and P'

Difference in node 6

$T' = T' \cup \{t_2\}$

Traverse t_3 in P and P'

Difference in node 9

$T' = T' \cup \{t_3\}$

Finish: $T' = \{t_2, t_3\}$

Wrapping up

Summary

Regression testing techniques are vital for testing software *as it evolves*.

- The size and complexity tend to increase.
- Test suites tend to expand, and require longer times to run.
- The pace of change can increase, with more developers making more changes to the system.

There are three broad families of regression testing techniques.

- Test suite minimisation: Try to remove redundant test cases
- Test suite prioritisation: Try to impose an order on tests so that the most important ones are executed early on.
- Test case selection: Try to pick test cases that are specific to a change in the code.

References

About the growth of test cases as software evolves, and the expense of executing them.

Memon, Atif, et al. "Taming Google-scale continuous testing." 2017 IEEE/ACM 39th International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP). IEEE, 2017.

Miranda, Charles, Guilherme Avelino, and Pedro Santos Neto. "Test Co-Evolution in Software Projects: A Large-Scale Empirical Study." Journal of Software: Evolution and Process 37.7 (2025): e70035.

Minimisation, Prioritisation, and Selection references:

Harrold MJ, Gupta R, Soffa ML. A methodology for controlling the size of a test suite. ACM Transactions on Software Engineering and Methodology (TOSEM) 2(3):270–285 (1993)

Rothermel G, Untch RH, Chu C, Harrold MJ. Prioritizing test cases for regression testing. IEEE Transactions on Software Engineering (TSE) 27(10):929-948 (2001)

Rothermel G, Harrold MJ. A safe, efficient regression test selection technique. ACM Transactions on Software Engineering and Methodology (TOSEM) 6(2):173–210 (1997)